Bolt-on, Verifiable Provenance for LLM-Powered Data Processing

Yiming Lin
University of California, Berkeley
yiminglin@berkeley.edu

Sepanta Zeighami
University of California, Berkeley
zeighami@berkeley.edu

Aditya G. Parameswaran
University of California, Berkeley
adityagp@berkeley.edu

arXiv:2608.25210v2 [cs.DB] 28 Aug 2026

ABSTRACT

Large Language Models (LLMs) are powerful tools for processing data. However, LLMs are also complex black-boxes, returning answers to queries on data, without any indication for where the answer came from or whether it is trustworthy. We introduce the notion of *provenance* for data processing with LLMs. While existing heuristics (such as embedding similarity or directly asking an LLM) could provide some hints for where the answer was derived, they provide no guarantees that the answer can be derived using the identified provenance, and indeed, are often incorrect. Instead, we propose the notion of *verifiable provenance* wherein we identify a subset of the input text that reproduces the same (or equivalent) answer as that on the complete text, and introduce the notion of *minimality*, where the verifiable provenance is as small as possible. To identify such a provenance, a naive solution would require checking all possible subsets of the source data with the LLM, which is prohibitively expensive. We present BLIP, a bolt-on framework for efficiently inferring a small-sized verifiable provenance for any LLM-powered data processing task, with any LLM. As part of BLIP, we introduce eight strategies, each guaranteed to find a minimal verifiable provenance, as well as an adaptive strategy that combines their strengths to reduce cost further. We further extend BLIP to produce multiple minimal verifiable provenances. Experiments on seven datasets show that the provenance generated by BLIP is always guaranteed to reproduce the answer—achieving over 30% higher accuracy than the best-performing baseline with a comparable provenance size. Moreover, BLIP incurs a low cost, comparable to the original query on the original data.

PVLDB Reference Format:
Yiming Lin, Sepanta Zeighami, and Aditya G. Parameswaran. Bolt-on, Verifiable Provenance for LLM-Powered Data Processing. PVLDB, 19(11): 2992 - 3005, 2026.
doi:10.14778/3836663.3836668

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/yiminl18/BLIP.git.

1 INTRODUCTION

Large Language Models (LLMs) have shown impressive capabilities in understanding, processing, and generating data, driving a wide range of data processing tasks, including data extraction, transformation, cleaning, and enrichment, as well as question-answering [13, 32, 47, 65, 67] over structured [34, 48] and unstructured data [68]. Recent declarative systems [36, 42, 44, 49–51, 53, 55, 57, 59], as well as SQL extensions to data warehouses [6, 21, 24, 60] enable LLM-powered operators over textual columns, supporting similar tasks such as extraction, cleaning, and synthesis. Most such LLM-powered data processing tasks or operators boil down to using an LLM to synthesize an answer for a given question from source data provided in-context, which could include structured or unstructured data. For example, data extraction can be framed as a natural language question, with the source data (documents or tables) provided in context. Or, in tabular question answering (TableQA) [34, 48], the question is placed in the prompt, with the relevant tables provided in context. In the following, we refer to this context or source data as *text*, since both structured and unstructured data are converted into a textual form for use as context, comprising one or more *sentences*, and to the data processing task or operator, expressed in natural language, as a *question*.

When using LLMs for data processing, it is *not clear how the LLM response may have been derived, and if the response can be trusted*. Although LLM capabilities continue to improve, they are far from perfect [46, 64], often with accuracy less than 75% in domains such as law, medicine, and finance [7, 29, 33, 41, 45]. Gaining human trust in how answers are produced and whether those answers are correct can be critical, especially when a wrong answer may lead to serious consequences. Consider a real-world use case: analysis of police use-of-force records, from the Police Records Access project, a collaboration that we are part of, along with various journalism and legal partners [2] to build the first-ever statewide use-of-force database[1]. As in Figure 1, journalists aim to extract names of officers involved in the use of force, from a 35-page document, and repeat this for over 3500 cases in the project, spanning over 1.5M pages [2]. The LLM `gpt-4o-mini` returns the answer *Officer Anderson, Maya*. Even if the LLM has high overall accuracy, that does not suffice to determine correctness for this specific query, where errors are especially likely with large context sizes [8, 46]. To ensure correctness, our journalism collaborators often manually review each text document (of dozens of pages). Here, an engineer on the project developed a custom keyword search approach to identify potentially relevant text portions for the answer (e.g., mentions of "Officer", "Captain"), reducing review time by 7×, but introducing many false positives and negatives. Overall, as the text size increases, verifying LLM answers becomes increasingly challenging. Ensuring human trust in answers is critical across domains, including analysis over medical, legal, financial, scientific, governmental, and journalistic documents. Effective mechanisms for verifying answer correctness are key to ensuring usability of LLM-powered applications.

In this paper, we revisit a traditional database concept in the context of verification of LLM-generated answers: *provenance*, which,

[1] Available at: https://clean.sfchronicle.com or https://clean.latimes.com

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 19, No. 11 ISSN 2150-8097.
doi:10.14778/3836663.3836668

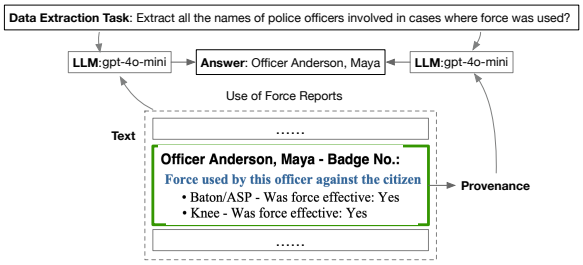


Figure 1: Verifiable Provenance in Police Use of Force Records. (Actual values in police records have been replaced for privacy.)

Method	Configuration	Qasper	NL_DEV	HotpotQA	CUAD	PubMedQA
Retrieval	RAG-1%	0.19	0.17	0.12	0.23	0.09
	RAG-5%	0.46	0.41	0.35	0.39	0.24
	RAG-10%	0.78	0.69	0.55	0.62	0.49
LLM	gpt-4o-mini	0.62	0.68	0.65	0.59	0.71
	gpt-4o	0.65	0.77	0.79	0.71	0.76
	gemini-2-flash	0.41	0.37	0.61	0.55	0.68

Table 1: Accuracy of retrieval-based and LLM-generated provenance.

for now, we will informally refer to as a text portion from which the answer may have been derived. In the use of force analysis in Figure 1, if supporting *evidence* is provided, such as a fragment of the document that mentions the use of force types and their associations with the officer's name, users can understand how the answer may have been generated and better assess its correctness.

Current approaches to infer provenance are not reliable. To infer the provenance of an answer, one could leverage retrieval approaches by identifying the top-k sentences in the given text with the highest relevance scores to the question [18, 22, 52]. These scores can be computed post hoc, using embedding similarities [22] or next-token probabilities [18]. However, these approaches do not guarantee that the provenance is sufficient to reproduce the answer. Table 1 shows the accuracy of one such retrieval method based on embedding similarity, i.e., via RAG [40], across five workloads: `Qasper` [20], `NL_DEV` [38], `HotpotQA` [63], `CUAD` [30], and `PubMedQA` [35], each with 500 question-document pairs. We vary the size of retrieved context (i.e., the inferred provenance) between 1%, 5%, and 10% of the full text. To assess whether the retrieved context can reproduce the answer, we ask the same question to the same LLM (gpt-4o) with this context, and report accuracy as the fraction of cases where the resulting answer agrees with the one generated from the full text.[2] Unfortunately, *even when the retrieved context includes 10% of the full text, accuracy is below 0.8*, making it neither reliable nor effective for human review.

Another method is to directly ask the LLM to identify the provenance of its answer from the text, by prompting it to provide the line numbers of the relevant sentences (by prefixing each sentence with line numbers), as listed in `LLM-provenance-prompt` below. Unfortunately, as shown in Table 1, `gpt-4o-mini`, `gpt-4o`, and `gemini-2-flash` return provenance that reproduces the answer in only 65%, 74%, and 53% of the cases, respectively—showing that LLMs are unable to identify the provenance that can reproduce the same (or equivalent) answer. When a provenance fails to reproduce the answer (e.g., those generated by retrieval or LLMs), human verification defaults to reading the entire document, significantly increasing human effort.

[2]If the answers aren't identical, we use LLM-as-a-judge [28, 66] to determine accuracy, which is known to be effective at identifying semantically equivalent answers.

`LLM-provenance-prompt`: [Question], [Answer], and [Context] are placeholders for question Q, answer A, and the full text T.

```
LLM-Provenance-Prompt: Given the following question:
[Question], the corresponding answers are: [Answer]. Your
task is to extract the set of sentences from the provided
context that contribute to generating these answers.
Identify the most relevant sentences that support the
given answers. Do not add explanations. Only return a list
of sentence IDs. Do not return any words. The context is
as follows: [Context]
```

Finally, other approaches [39] require access to the model to compute attribution scores for tokens based on training data but rely on access to training data, which is unavailable for black-box LLMs (e.g., OpenAI models) and frequently withheld for open-source models (e.g., Llama3). They also don't apply when operating on user-provided text, which is not part of the training data.

Verifiable provenance. We use insights from the previous experiment to introduce the concept of *verifiable provenance* over text for LLM-powered data processing tasks. Instead of arbitrarily selecting a set of sentences based on their relevance to the question and answer, we require the provenance to be *verifiable*, i.e., the provenance is a subsequence of the text (e.g., green box in Figure 1) that can *reproduce the same (or equivalent) answer as the one obtained from the full text when queried using the same LLM*. In the example in Figure 1, the provenance in the green box is a subsequence of the original text, and asking the same question on just this provenance with the same LLM `gpt-4o-mini` leads to the same answer, *Officer Anderson, Maya*. Instead, the results in Table 1 show that roughly *half of the LLM-generated provenance is not verifiable.*

Conciseness of verifiable provenance and its challenges. To help users easily assess answer correctness, the verifiable provenance must additionally be *small-sized*. Otherwise, the original text could itself serve as a provenance, but reviewing this is difficult and time-consuming. Identifying a verifiable provenance that is also concise is nontrivial. One naive approach would involve considering all subsets of $k \in [0, n]$ sentences of the text with n sentences and verifying whether the enumerated subset produces an equivalent answer with the LLM, which would scale as 2^n. Another approach would start from the given text as the initial provenance, and iteratively remove a sentence at a time if the remaining still produces the same answer when using the same LLM—until no further sentences can be removed. However, this approach incurs significant cost and latency, as it requires a number of LLM calls proportional to the number of sentences in the text. This baseline, on the Qasper dataset with 9,000 tokens on average, takes >2 hours, incurring >100× the cost of using an LLM for answering the question on the original text.

Merely a BLIP to identify verifiable, concise provenance. We introduce BLIP[3], *a provenance discovery framework comprising a family of intuitive strategies to efficiently infer small-sized verifiable provenance*. We define the notion of *minimality* of verifiable provenance, where removing any sentence from the provenance results in a different answer. We further define the notions of *strong* and

[3]Short for **B**olt-on **L**LM ver**I**fiable **P**rovenance.

weak monotonicity to characterize the types of tasks for which minimal provenance provides additional guarantees, while empirically showing that those tasks dominate our workloads—and contrast these notions to those in the provenance literature [11, 17, 23, 26].

As part of BLIP, we present a two-phase approach that is *guaranteed to find a minimal verifiable provenance*. The first phase aggressively returns a small-sized verifiable provenance that may not be minimal. In this phase, we explore four strategies, including ranking text portions based on their relevance to the question and answer, as well as different enumeration orders. The second phase takes the provenance returned from the first phase and guarantees a minimal provenance. This two-phase approach, though intuitive, exhaustively explores 10 strategy combinations, each producing a minimal provenance. Using a cost model that we introduce, we present a theoretical cost comparison, and propose an adaptive strategy that integrates their best features. We further employ KV-caching [5, 14, 43] to optimize each individual strategy to minimize inference cost. Finally, we extend BLIP to support the discovery of multiple minimal provenances rather than just one, to support use cases where multiple pieces of evidence are helpful.

Overall, BLIP can be applied in a bolt-on manner, is compatible with any LLM model, and offers a theoretical guarantee of minimal provenance at a low cost. Our contributions include:

- We *are the first to introduce the notion of verifiable provenance for LLM-powered data processing*, outline key properties, including verifiability, minimality, and monotonicity, and characterize the family of tasks where verifiable provenance applies.
- We conduct an exhaustive exploration of intuitive but effective strategies to efficiently infer minimal provenance at low cost.
- We present a theoretical analysis of correctness and cost of the strategies, and use it to develop an adaptive strategy.
- We perform comprehensive evaluations across seven datasets, demonstrating that it is possible to infer minimal provenance with **guaranteed 100% accuracy**, achieving **over 30%** higher accuracy than the best-performing baseline, at a low cost—comparable to that of performing the original task.

We define the notion of provenance and present an overview of BLIP in Section 2. We then present our approaches for finding a single minimal provenance and top-k minimal provenances in Section 3 and Section 4, respectively. We evaluate BLIP in Section 5.

2 PROBLEM DEFINITION AND OVERVIEW

We begin by formally defining the concept of provenance (Section 2.1) and outlining our cost model (Section 2.2), followed by a description of our problem statement (Section 2.3). We further define the applicable tasks (Section 2.4) and questions (Section 2.5) for which our provenance is useful, and discuss the impact of LLM behavior (e.g., non-determinism) on provenance (Section 2.6). We finally present an overview of BLIP (Section 2.7).

2.1 Provenance Definition

Consider a data processing task generally represented as a natural language *question* Q. Let L be the Large Language Model (LLM), and T be the input *text*, i.e., source data to process. T consists of a sequence of sentences, i.e., $T = \langle s_1, s_2, \ldots, s_n \rangle$, where i is the *index* of sentence s_i. Let A be the *response* (or answer) to question Q in text T using LLM L, denoted as $L(T, Q) = A$. Let $P \sqsubseteq T$ be a subsequence of T, i.e., $P = \langle s_{i_1}, s_{i_2}, ..., s_{i_m} \rangle$, where $1 \leq i_1 < ... < i_m \leq n$.

Definition 1. Verifiable Provenance. Given text T, question Q, and LLM L, where $L(T, Q) = A$, we define the subsequence $P \sqsubseteq T$ to be a *verifiable provenance* if $L(P, Q) = A'$, where $I(A, A') = \text{True}$.

The indicator function $I(A, A')$ returns *True* if A and A' are lexically identical or semantically equivalent. For settings where A is drawn from a closed domain (e.g., true/false or zip code), users can opt for $I(A, A')$ as $A{=}A'$. Otherwise, we leverage an LLM as a judge [28, 66] to evaluate semantic equivalence (e.g., "2012" and "year 2012" judged equivalent). This approach effectively handles equivalent but non-identical LLM-generated responses. To show this, we conducted an evaluation on 2000 text pairs from the GLUE benchmark [58], where each pair contains two text snippets with a label indicating equivalence. We evaluated LLM-as-a-judge using five LLMs with different capabilities and four prompts, including a handwritten prompt and an LLM-rewritten variant, each with and without examples. . Details of prompts and experimental setup can be found below. As shown in Table 2, LLM-as-a-judge is effective with around 94% accuracy on average across models. The LLM used to evaluate $I(A, A')$ does not need to be the one used to answer Q on T.

response_equal_human.: [sentence_1] and [sentence_2] are placeholders for two test sentences.

```
llm_equal_human: Are the following two sentences semantically equivalent? Please respond with True if they are, and False if they are not. Sentence 1: [sentence_1]. Sentence 2: [sentence_2].
```

response_equal_human_example.: [sentence_1] and [sentence_2] are placeholders for two test sentences.

```
llm_equal_human_example: Are the following two sentences semantically equivalent? Please respond with True if they are, and False if they are not. Examples: Example 1: Sentence 1: The cat is sleeping on the sofa. Sentence 2: A cat is lying on the couch asleep. Answer: True Example 2: Sentence 1: The company reported a profit of 2 million dollars. Sentence 2: The company reported a loss of 2 million dollars. Answer: False Sentence 1: [sentence_1]. Sentence 2: [sentence_2].
```

response_equal_llm.: [sentence_1] and [sentence_2] are placeholders for two test sentences.

```
llm_equal_2: Determine whether the following two sentences are semantically equivalent. Return True if they are, and False otherwise. Are the following two sentences semantically equivalent? Two sentences are semantically equivalent if they express the same meaning, even if the wording is different. Minor paraphrasing, reordering, or synonym replacement does NOT change equivalence. Differences in factual content, numerical values, negation, or core claims mean they are NOT equivalent. Sentence 1: [sentence_1]. Sentence 2: [sentence_2].
```

response_equal_llm_example.: [sentence_1] and [sentence_2] are placeholders for two test sentences.

	gpt-4o	gpt-4o-mini	gpt-5	gpt-5-mini	gemini-2-flash
response_equal_human	96.3%	91.2%	94.2%	90.4%	94.7%
response_equal_human_example	96.6%	90.8%	95.1%	90.9%	94.3%
response_equal_llm	96.8%	90.8%	94.7%	91.1%	95.2%
response_equal_llm_example	96.4%	91.5%	93.4%	91.6%	95.7%

Table 2: Effectiveness of LLM-as-a-judge.

```
llm_equal_2: Determine whether the following two sentences
are semantically equivalent. Return True if they are,
and False otherwise. Are the following two sentences
semantically equivalent? Two sentences are semantically
equivalent if they express the same meaning, even
if the wording is different. Minor paraphrasing,
reordering, or synonym replacement does NOT change
equivalence. Differences in factual content, numerical
values, negation, or core claims mean they are NOT
equivalent. Examples:
Example 1: Sentence 1: She did not attend the meeting.
Sentence 2: She skipped the meeting. Answer: True
Example 2: Sentence 1: The experiment was conducted
in 2020. Sentence 2: The experiment was conducted in
2021. Answer: False Sentence 1: [sentence_1]. Sentence
2: [sentence_2].
```

Intuitively, any subsequence of T that returns (lexically or semantically) equivalent answer to A using the same LLM is considered a verifiable provenance. When obvious from context, we refer to a verifiable provenance simply as *provenance*. In practice, users prefer the provenance to be small (i.e., with fewer sentences) for easier interpretation. Thus, we define the notion of a *minimal provenance*.

Definition 2. Minimal Provenance. A provenance P is *minimal*, if $\forall s \in P, I(L(P \setminus s, Q), A) = False$.

That is, a provenance P is minimal if removing any sentence from P results in an answer that is not equivalent to A.

Definition 3. Strictly-Minimal Provenance. A provenance P is *strictly minimal*, if $\nexists P' \sqsubset P, I(L(P', Q), A) = True$.

A provenance P is strictly minimal if there does not exist any subsequence of P that is a provenance. Any strictly minimal provenance is also minimal, by definition. In practice, a minimal provenance is often sufficient, as a small-sized, rather than strictly minimal, provenance is typically adequate for interpreting answers. As we will show shortly, under a commonly observed assumption for LLMs, a minimal provenance is equivalent to a strictly minimal one.

2.2 Cost Model and KV-Cache

Let $PT = \langle Q, T \rangle$ denote the prompt formed by concatenating the question Q with the text T. We use $L(T, Q)$ and $L(PT)$ interchangeably throughout the paper when it does not lead to confusion. Let c_{PT} represent the monetary cost of a single LLM invocation of L with input PT, returning a response $L(PT)$. We have:

$$c_{PT} = |PT| \cdot c_{in} + |L(PT)| \cdot c_{out} \quad (1)$$

Here, $|\cdot|$ denotes the number of tokens, and c_{in}, c_{out} refer to the unit cost of model L for input and output tokens, respectively. For example, when L is `gpt-4o-mini`, c_{in} is $\$\frac{0.15}{10^6}$ and c_{out} is $\$\frac{0.6}{10^6}$.

If two prompts PT_i and PT_j share a prefix, denoted $Pre_{i,j}$, then $Pre_{i,j}$ is considered cached tokens, employing the *KV-cache* [5, 14, 43]. For example, with `gpt-4o-mini` and `gemini-2-flash`, cached tokens are 2× and 4× cheaper than uncached tokens, respectively. In particular, if the prompt PT_i has already been computed using L, then the cost of inference on PT_j, denoted $c_{PT_j|PT_i}$, is

$$c_{PT_j|PT_i} = \frac{|Pre_{i,j}| \cdot c_{in}}{f_L} + |PT_j \setminus Pre_{i,j}| \cdot c_{in} + |L(PT_j)| \cdot c_{out} \quad (2)$$

Here, f_L is the cost reduction factor for cached tokens in model L. For example, $f_{\text{gpt-4o-mini}} = 2$, while $f_{\text{gemini-2-flash}} = 4$.

LLMs	Task	Qasper	NL_DEV	HotpotQA	Average
gpt-4o-mini	SM	97%	87%	88%	90.7%
	WM	99%	96%	98%	97.7%
gpt-4o	SM	98%	93%	91%	94.0%
	WM	100%	98%	98%	98.7%
gemini-2-flash	SM	95%	88%	81%	88.0%
	WM	97%	96%	97%	96.7%

Table 3: Percentage of Strongly-Monotonic (SM) and Weakly-Monotonic (WM) Tasks, with per-row averages.

2.3 Problem Definitions

We are now ready to state our problem more formally:

Problem 1. Provenance Inference. Given text T, question Q, LLM L and answer $A = L(T, Q)$, identify a minimal provenance.

The minimal provenance in Definition 2 for $\langle Q, A \rangle$ within text T may not be unique. In our example in Figure 1, the same officers may be listed multiple times, in the internal affairs report, court hearings report,as well as incident summary. Let $MP = \{mp_1, \ldots, mp_n\}$ be the set of distinct minimal provenances within T. Although in most cases, identifying a single minimal provenance $mp_i \in MP$ is sufficient to provide evidence for human verification, there are scenarios where multiple minimal provenances may be helpful, as we will outline in Section 4. We therefore define the *k-Provenance Inference* problem below.

Problem 2. k-Provenance Inference. Given text T, question Q, LLM L, and answer $A = L(T, Q)$, identify k distinct minimal provenances, $\{mp_{i1}, mp_{i2}, ..., mp_{ik}\}$, where $\forall ij, mp_{ij} \in MP$.

2.4 Strong and Weak Monotonicity

As described previously, our goal, depending on the problem, is to find one or k minimal provenances. There is a danger, however, of getting stuck at a "local minimum", where the provenance P returned is minimal, but there is a subsequence $P' \sqsubset P$ that is also a provenance—a scenario that is addressed by strict minimality. Here, we introduce two properties and empirically study their adherence for our tasks. We denote a task by the triple $\langle Q, T, L \rangle$.

Definition 4. Strongly Monotonic Tasks. A task $\langle Q, T, L \rangle$ with $L(T, Q) = A$, is *strongly monotonic* if for any provenance $P \sqsubset T$, $\forall P'$ where $\forall P \sqsubset P' \sqsubset T$, P' is also a provenance.

A task is *strongly monotonic* if any superset of any provenance is also a provenance. While this may appear restrictive, we empirically show in Table 3 that strongly monotonic tasks are still common, around 90% on average across real workloads with multiple LLMs. (Experimental setup is introduced later.) We next introduce the notion of *weakly monotonic* tasks.

Definition 5. Weakly Monotonic Tasks. A task $\langle Q, T, L \rangle$ with $L(T, Q) = A$ is *weakly monotonic* if for any provenance $P \sqsubset T$, where $|P| = i$ and $|T| = n$, there is a sequence of provenances $P_{i+1}, P_{i+2}, ..., P_{n-1}$ such that $|P_{i+k}| = i + k$, and $P \sqsubset P_{i+1} \sqsubset P_{i+2} \sqsubset ... \sqsubset P_{n-1} \sqsubset T$.

Instead of requiring that every superset P' of P is a provenance as in strong monotonicity, as shown in Figure 2, a weakly monotonic task simply requires a *chain of provenances*, covering every distinct

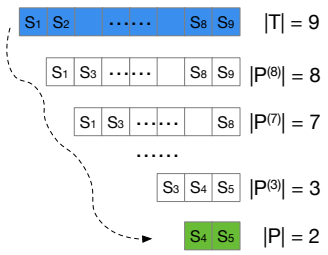


Figure 2: Weak Monotonicity.

size from $|P|$ to $|T|$, where each larger provenance is a superset of smaller ones. Weak monotonicity is useful because it implies the existence of a *path* to a minimal provenance (e.g., $\langle s_4, s_5 \rangle$) from T (e.g., $\langle s_1, \ldots, s_9 \rangle$). We find that *weakly monotonic tasks are common, with a prevalence of* **nearly over 95%** *across workloads and LLMs.*

We finally connect our notion of weak monotonicity with the notion of minimal provenance, showing that, for weakly monotonic tasks, a minimal provenance is also strictly minimal.

Theorem 1. For a weakly monotonic task $\langle Q, T, L \rangle$, all minimal provenances $P \sqsubseteq T$ are also strictly minimal.

The proof can be found below. As we will see later, all our solutions for Problem 1 are guaranteed to return a minimal provenance for *any* task. Theorem 1 ***additionally guarantees such minimal provenance is also strictly minimal***, as long as the task is weakly monotonic— the dominant class of tasks in our workload.

Proof. A strictly minimal provenance is also a minimal provenance by definition. Next, we prove that for a weakly monotonic task $\langle Q, T, L \rangle$, a minimal provenance $P \sqsubseteq T$ is also a strictly minimal provenance. Assume there exists a $P' \sqsubset P$ such that P' is a provenance. We will prove that such a P' does not exist, thereby establishing the correctness of the theorem by contradiction. Let $|P| = n$ and $|P'| = i$. Since P is a minimal provenance, $\forall s \in P, P \setminus s$ is not a provenance. Thus, $|P'|$ must be less than $n-1$. Since the task $\langle Q, T, L \rangle$ is weakly monotonic, there must exist a set of provenances from P to P', i.e., $P' \sqsubset P_{i+1} \sqsubset \cdots \sqsubset P_{n-1} \sqsubset P$, where provenance P_{i+1} has size $|i+1|$. Since $\forall s \in P, P \setminus s$ is not a provenance, $\nexists P_{n-1} \sqsubset P$, which contradicts the weak monotonicity property. This completes the proof by contradiction. □

So far we have defined strongly and weakly monotonic tasks. We now briefly describe the experimental setup for the results in Table 3.

Monotonicity testing protocol. To compute the percentage of strongly monotonic tasks, for each document-question pair, we begin with a minimal provenance mp. We then randomly sample 100 supersets of mp, with sizes uniformly distributed between $|mp| + 1$ and $|T| - 1$. We compute the percentage of supersets that reproduce an answer equivalent to A, and average this value across all document-question pairs for each dataset.

To compute the percentage of weakly monotonic tasks, we similarly start from a mp. Let $|mp| = i$ and $|T| = n$. Let $\mathcal{P} = \{P_{i+1}, P_{i+2}, ..., P_{n-1}\}$ be a set of provenances such that $|P_{i+k}| = i + k$, and $P \sqsubset P_{i+1} \sqsubset P_{i+2} \sqsubset ... \sqsubset P_{n-1} \sqsubset T$. We perform a greedy exhaustive search to construct $\mathcal{P}$. Starting from mp, we enumerate candidates for P_{i+1} and stop once one reproduces an answer equivalent to A. We then search for P_{i+2}, a superset of P_{i+1}, and continue this process. If no superset of P_{i+1} with a size of $|i + 2|$ yields a valid provenance, we backtrack to explore alternative candidates for P_{i+1}. This continues until $\mathcal{P}$ is discovered or if not, we treated this as a non-weakly-monotonic task, and we terminate it once the number of LLM-based tests exceeds $m = 500$ to avoid potentially high cost.

	Qasper		NL_DEV		HotpotQA	
	str_s	llm_s	str_s	llm_s	str_s	llm_s
gpt-4o-mini	0.94	**0.97**	0.97	**0.99**	0.99	**0.995**
gpt-4o	0.93	**0.99**	0.97	**0.998**	0.99	**0.998**
gemini-2-flash	0.97	**0.99**	0.96	**0.97**	0.99	**0.99**

Table 4: LLM Deterministic Evaluation.

2.5 Provenance Correctness

We just discussed minimality properties of verifiable provenance, but have not yet confronted whether the provenance matches user intent. Recall that a verifiable provenance is a subset of the input that is sufficient for an LLM to produce an answer as it would on the whole input. This definition, however, does not imply that the verifiable provenance is sufficient for a human to arrive at the same answer as the LLM. We call a provenance that enables a human to arrive at the same answer as the LLM a *true provenance*.

We first discuss two scenarios where a true provenance is undefined and thus our provenance is inapplicable. First, if the question is ambiguous, then the LLM's answer is not meaningful, making the provenance inapplicable. For example, when asking for "*the revenue of Amazon*" over its financial report, it is unclear whether the question refers to the monthly, quarterly, or yearly revenue. Second, if the LLM's answer (on the entire input) is incorrect, then any provenance is incorrect by definition—no provenance is sufficient for a human to arrive at the same answer as the LLM.

When a true provenance is well-defined, a verifiable provenance is *correct* if it matches the true provenance. We next provide a taxonomy of cases where a verifiable provenance can be incorrect (Figure 3). Since the true provenance is unknown, this taxonomy provides a way to understand when to apply our techniques.

Unstable provenance. We say that our verifiable provenance is *unstable* when it is not meaningful for a given task, so the returned provenance can be incorrect even if the LLM answer is correct. We now discuss set of questions Q for which verifiable provenance is stable. As in classical provenance literature [11, 17, 23, 26], Q can represent the computation corresponding to any *positive relational algebra* ($\mathcal{RA}^+$); this class encapsulates SPJRU (Select-Project-Join-Rename-Union) queries, along with recursion. This broad class of queries encompasses data extraction (e.g., "extract all the dates in this text") or transformation (e.g., "reformat the intake form as a table"). Q could also include multi-hop reasoning (e.g., "what was the origin of the money that was given to the officer?), analogous to a recursive query that follows exchanges of money to its source. Prior work on *why* and *where* [11] and *how* [26] provenance also focus on $\mathcal{RA}^+$. In fact, Glavic [23] describes that why provenance is synonymous with $\mathcal{RA}^+$. Q could also include as part of its output computation corresponding to monotonic aggregates, i.e., where the aggregates trend in a single direction, such as `COUNT(*)` or `SUM` of non-zero positive numbers; e.g., "count the number of officers involved" or "summarize all medical aspects in the document".

As in traditional provenance, as soon as Q includes arbitrary negation or aggregation, or operations on aggregates, the usefulness of a minimal provenance decreases. As an extreme example, if Q represents the computation of the average of numbers $\in \{1, 2, 3, 4, 5\}$, one minimal provenance could be $\{3\}$, which is not very useful. Likewise, if Q includes a predicate on an aggregate, e.g., "find all

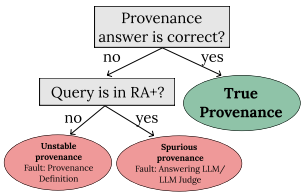


Figure 3: Provenance Correctness Taxonomy

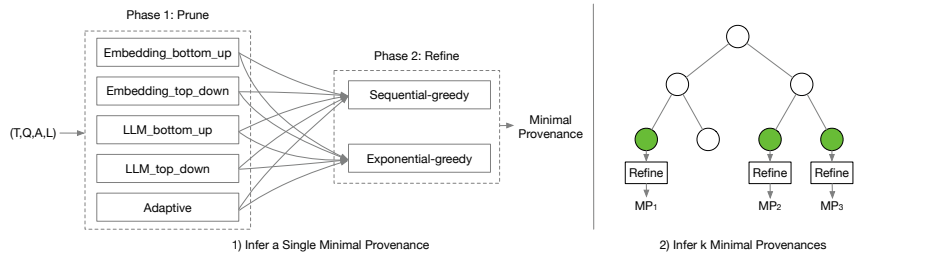


Figure 4: Overview of BLIP. T, Q, A, and L denote the input text T, natural language question Q, answer (as evaluated) A, and the LLM model used L, respectively. MP stands for minimal provenance.

officers who were mentioned at most three times", a minimal provenance might list just one mention for each output officer, even if they were listed twice or three times, which may not be useful.

Spurious provenance. Finally, we say that a verifiable provenance can be *spurious* even for $\mathcal{RA}^+$, when LLMs make incorrect judgments. For example, it may incorrectly judge that a true provenance fails to reproduce the answer on the entire input.

2.6 Handling Non-Determinism in LLM

So far, we have defined our problems, the applicable tasks based on monotonicity, and the class of questions for which provenance is useful. We now examine LLM behavior and its impact on provenance. When answering questions, instead of producing a single output, LLMs produce outputs with a probability distribution, which might introduce uncertainty when the same input is processed across multiple runs. While this uncertainty is already handled by our use of LLM-as-a-judge to accomodate for non-identical, but equivalent answers, here we go further to see how LLM non-determinism can impact provenance.

Overall, the degree of uncertainty in LLM output depends on the decoding strategy. When greedy decoding is used (by setting the temperature to 0), the model picks the most probable token at each step [1], and is thus theoretically deterministic. To validate this, we run each of 500 question–text pairs from our three workloads 50 times. We then cluster the 50 outputs per pair using exact string match (resp., LLM-as-a-judge), and report the maximum cluster size divided by 50 as *str_s* (resp., *llm_s*) in Table 4. Non-deterministic outputs measured by string match happen about 1% to 7% of the time across workloads. Such a gap occurs when probabilities are tied, due to factors like non-deterministic GPU kernels, quantization rounding, or floating-point errors [3]. These cases are caught by LLM-as-a-judge, resulting in **deterministic outputs >99%** of the time—so the impact of non-determinism is insignificant.

To ensure determinism on all possible inputs, we further extend the indicator function $I()$ (in Definition 1) that checks if two LLM outputs are equivalent to handle the special case of output probabilities being tied. Here, instead of considering a single LLM output, let $\mathcal{A}$ and $\mathcal{A}'$ be the set of all LLM outputs on the full text T and a subset of the text P. Then, to verify whether P is a verifiable provenance, we check if the outputs with the maximum probability in $\mathcal{A}$ and $\mathcal{A}'$ are equivalent. We introduce a small constant ($\epsilon = 0.01$ in our implementation) to account for floating-point errors when identifying outputs with the maximum probability. For example, consider $\mathcal{A} = \{a_1, a_2, a_3\}$ (where a_i is a possible output) with corresponding probabilities 0.43, 0.425, and 0.145. In this case, both a_1 and a_2 are outputs with maximum probability under ϵ-relaxation. $\mathcal{A}$ is equivalent to $\mathcal{A}' = \{a'_1, a'_2\}$ (where a'_1 has a probability of 0.95) if either $I(a_1, a'_1)$ or $I(a_2, a'_1)$ returns True. In the following, we adopt this implementation of $I()$, which, however, has a negligible impact on our results, occurring in less than 1% of cases, compared to the original $I()$ implementation.

2.7 Overview of BLIP

Our notion of provenance considers a "black-box" LLM that performs an unknown computation on a text data source whose output we observe. The goal of our provenance-finding algorithm is to identify the minimal text that can lead to an equivalent output. Because the underlying computation is unknown, our approach, unlike prior work on provenance in relational databases [11, 17, 26], cannot track provenance throughout query evaluation. Instead, we develop a method that *reverse-engineers* the LLM's answering process after the fact.

We next present an overview of BLIP, our provenance discovery framework, as shown in Figure 4. Given text T, question Q, LLM L, and answer $A = L(Q, T)$, BLIP uses a two-phase approach to infer a single minimal provenance (Section 3), and can be extended to find k distinct minimal provenances (Section 4).

Finding a single minimal provenance. BLIP uses a two-phase approach, *prune-refine*, to return a minimal provenance. The *pruning* phase returns a small-sized provenance by aggressively pruning out sentences that likely do not contribute to A, while the *refinement* phase takes the provenance returned from the pruning phase and refines it to obtain a minimal provenance.

For the pruning phase, we propose four strategies to reduce provenance size by leveraging *text relevance to Q and A* and *different scan orders* (top-down or bottom-up).

- (Text Relevance) We divide the text T into equal-sized portions and prioritize examining those with high relevance to the pair $\langle Q, A\rangle$, estimated using embedding similarity [15] or an LLM.
- (Scan Orders) To return a small-sized provenance quickly, one approach is to process text portions in decreasing order of relevance to $\langle Q, A\rangle$, incrementally add a portion at a time, and return the top-i portions as soon as they yield an answer equivalent to A. Alternatively, starting from T, we repeatedly discard the least relevant half until the remaining portions fail to reproduce A.

As bottom-up and top-down approaches excel in different scenarios, we provide a theoretical cost analysis to characterize where each shines, based on the size of the returned provenance. We further propose an adaptive method that combines them to robustly infer a small-sized provenance while avoiding their worst-case limitations.

In the refinement phase, we propose two strategies that take any provenance (e.g., the output from the prune phase) and return a corresponding minimal provenance by removing all non-relevant sentences with respect to $\langle Q, A\rangle$. The first strategy, *sequential-greedy*, iteratively deletes sentences from a given provenance if the remaining text after each removal still produces an answer equivalent to A. In contrast to *sequential-greedy* that considers one sentence at a time, the second strategy, *exponential-greedy*, removes sentences in

exponentially increasing batch sizes. Starting with one sentence, if deletion succeeds, it attempts to remove two, then four, and so on, doubling the batch size each time to accelerate pruning. Both strategies enumerate sentences in decreasing order of their indexes (from larger to smaller) to maximize token reuse via the KV cache in each LLM invocation, leading to a significant cost reduction.

Find top-k minimal provenances. The provenance returned by the two-phase approach is minimal. However, in certain scenarios, as outlined in Section 4, additional evidence is needed. Thus, as part of BLIP, we introduce a greedy tree-search approach to return the k distinct minimal provenances, allowing users to explore all provenances and gain greater confidence in the answers.

3 FINDING TOP-1 PROVENANCE

Given the input text T, question Q, LLM L, and answer A, we present a two-phase approach to infer a single minimal provenance.

3.1 Phase 1: Provenance Pruning

To prune sentences that do not contribute to the answer A, we propose four intuitive strategies that leverage heuristics to rank sentences by their relevance to the $\langle Q, A\rangle$ pair (e.g., embedding similarities or LLMs), and different scan orders (e.g., bottom-up or top-down), along with an adaptive strategy that combines the strengths of these approaches in Section 3.3.

Strategy 1: Embedding-bottom-up. In this strategy (Algorithm 1), we divide the input text T into a list of equal-sized blocks, $T = \langle b_1, b_2, \ldots, b_m\rangle$, where each block contains a sequence of sentences (Line 1). The value m is chosen to balance the cost of strategies and the size of the returned provenance. (Empirically, setting $m = 20$ strikes a good balance, as shown in Section 5) We then sort the blocks in descending order of embedding similarity to $\langle Q, A\rangle$, i.e., the concatenation of Q and A (by setting `ranker` as `embedding` in Line 2). As soon as the first i sorted blocks reproduce an answer equivalent to A, we stop searching and return them as the provenance (Lines 5–9). When evaluating the top-i blocks using LLM L, we always re-order the sentences in the evaluated blocks in increasing order of their indexes in T, as implemented by `rerank_index` in Line 7. For example, the top-i blocks $\langle b_7, b_8, b_2, b_3\rangle$ are reordered as $\langle b_2, b_3, b_7, b_8\rangle$ before being passed to L for answer generation, to prevent unintended changes in meaning due to reordering.

Strategy 2: Embedding-top-down. This strategy instead performs a top-down search over the blocks sorted by embedding similarity, by setting `scan` to `top-down` in Line 10 of Algorithm 1. We start with the entire ranked list of blocks B from T as the initial provenance P, and perform binary search to identify the smallest top-k prefix that reproduces an answer equivalent to A. At each iteration, we consider the midpoint $mid = \lceil \frac{l+r}{2} \rceil$ of the current index range $[l, r]$, and evaluate the top-mid blocks, i.e., $<b_1, b_2, \ldots, b_{mid}>$ to see if they constitute a provenance (Line 11-15). If they reproduce an answer equivalent to A, we refine the search window to the left half $[l, mid - 1]$; otherwise, we consider the right half $[mid + 1, r]$ in the next iteration (Line 16-19). The algorithm makes at most $\lceil \log_2 m \rceil$ LLM calls, where m is the number of blocks in T.

Strategy 3 & 4: LLM-bottom-up and LLM-top-down. These strategies are like 1 & 2 but instead of sorting blocks based on embedding similarities, they invoke an LLM model L' to score each block b_i in B on a scale of 1 to 10 based on the relevance of b_i to the $\langle Q, A\rangle$ pair, using the `LLM-Ranker-Prompt` below. The model L' used to score the blocks does not necessarily need to be the same as the model L used to answer Q.

`LLM-Ranker-Prompt`: the [Q], [A], and [Text blocks] are placeholders corresponding to Q, A, and B in Algorithm 1.

Intuitively, given text blocks B sorted by embedding similarities or LLMs, when the provenance lies within the first-i blocks and i is small (i.e., the provenance size is small and the ranker is effective), a bottom-up search is ideal as it can quickly identify a small provenance. On the other hand, if i is large, bottom-up incurs high cost by repeatedly evaluating the first i blocks, resulting in a quadratic number of block accesses. In such a case, the top-down search is more efficient, as it discards blocks exponentially faster. In Section 3.3, we analyze the costs of both strategies and in Section 3.3.4 recommend what approach to use for a given dataset.

Algorithm 1: `Prune[ranker, scan]`

```
   Input: ⟨T, Q, L⟩
1  T ← ⟨b_1, b_2, ..., b_m⟩; A ← L(T, Q)
2  B ← rank(T, ranker)
3  P ← ∅
4  if scan == bottom-up then
5      for b_i ∈ B, in ascending order of i do
6          P ← P ∪ b_i
7          P' ← rerank_index(P)
8          if I(L(P', Q), A) ==True then
9              break
10 if scan == top-down then
11     l ← 1, r ← m
12     while l ≤ r do
13         mid ← ⌈(l+r)/2⌉
14         P ←< b_1, b_2, ..., b_mid >
15         P' ← rerank_index(P)
16         if I(L(P', Q), A) == True then
17             r ← mid − 1
18         else
19             l ← mid + 1
20 P ← rerank_index(P)
21 Return P
```

```
LLM-Ranker-Prompt: Given the following question: [Q], and
a list of text blocks, the corresponding answers are [A].
Your task is to assign a score (from 1 to 10) to each block
based on how likely it is to contain context relevant to
answering the question. The text blocks are listed below,
each starting with Block i: followed by its content. Return
only a comma-separated list of scores corresponding to
each block, in the order they are given. Do not include
any explanations or additional text. [Text blocks]
```

3.2 Phase 2: Provenance Refinement

The returned provenance from Phase 1, while smaller than T, may not be minimal. In this phase, we present two strategies that take as input any provenance and compute a minimal provenance.

Strategy 1: Sequential-Greedy. Taking any provenance P as input, `Sequential_Greedy` (in Algorithm 2) considers each sentence $s_i \in P$ in descending order of i to maximize the prefixes shared among LLM inferences. If after removing a sentence s_i from the current provenance P, the remaining text $P \setminus s_i$ produces an answer equivalent to A, then s_i is removed from P; otherwise, s_i is retained

Algorithm 2: `Refine[scan]`

```
   Input: P, ⟨T, Q, L⟩
1  Function Sequential_Greedy(P):
2      for s_i ∈ P, in descending order of i do
3          P' ← P \ s_i
4          if I(L(P', Q), A) == True then
5              P ← P'
6      Return P
7  Function Exponential_Greedy(P):
8      j ← |P| − 1, l ← 0
9      while j ≥ 0 do
10         i ← j − 2^l + 1
11         P' ← P \ P_[i,j]
12         if I(L(P', Q), A) == True then
13             P ← P'
14             j ← i − 1, l ← l + 1
15         else
16             l ← 0
17 if scan == sequential then
18     P ← Sequential_Greedy(P); P_prev ← P; A ← L(T, Q)
19     while P_prev ≠ P do
20         P_prev ← P
21         P ← Sequential_Greedy(P)
22 if scan == exponential then
23     P ← Exponential_Greedy(P); P_prev ← P; A ← L(T, Q)
24     while P_prev ≠ P do
25         P_prev ← P
26         P ← Exponential_Greedy(P)
27 Return P
```

(Lines 1–6). We repeatedly invoke `Sequential_Greedy` on the resulting provenance P until P no longer changes (Lines 18–21). This step (and a similar one in Strategy 2) is essential for minimality.

Example 1. Assume the provenance P returned from the first phase is $P = \langle s_1, s_2, s_3, \ldots, s_{20}\rangle$. In the first iteration, s_{20} will be removed if $P \setminus \{s_{20}\}$ returns an equivalent answer to A. Next, when s_{19} is examined, its prompt (consisting of the question and context) shares a prefix with the previous one since both contain the same question followed by the same prefix of context, i.e., $\{s_1, s_2, \ldots, s_{18}\}$. As a result, the cost of input tokens in the second iteration is reduced by a factor of $f_L = 4$ (or 2) when using `gemini-2-flash` (or `gpt-4o-mini`), respectively, because the entire prompt consists of cached tokens. Similar cost reductions apply to subsequent LLM inferences, resulting in significant cost savings. □

Strategy 2: Exponential-Greedy. In contrast to considering sentences one by one, `Exponential_Greedy` (Algorithm 2) removes sentences more aggressively by exponentially increasing the number of sentences considered for removal, as long as the result continues to produce an answer equivalent to A. Let $P_{[i,j]}$ denote a subsequence of P from the i-th to the j-th sentence within P.

We use a running example to explain the behavior. Suppose the provenance P returned from the first phase is $P = \langle s_1, s_2, s_3, \ldots, s_{20}\rangle$. This strategy considers each subsequence $P_{[i,j]}$ within P. In the first iteration, it considers removing a subsequence $P_{[20,20]} = \langle s_{20}\rangle$. If $P \setminus P_{[20,20]}$ produces an equivalent answer to A, then $P_{[20,20]}$ is removed from P. In the next iteration, instead of only s_{19}, we double the length of the candidate subsequence and consider removing $P_{[18,19]} = \langle s_{18}, s_{19}\rangle$. If the remaining text $\langle s_1, s_2, \ldots, s_{17}\rangle$ still produces an equivalent answer, $P_{[18,19]}$ is removed, and we proceed to $P_{[14,17]}$. If, in any iteration, the remaining text after removing the current subsequence fails to reproduce the answer, the length of the next candidate subsequence is reset to 1 (Lines 8–16). For example, if removing $P_{[18,19]}$ causes the remaining text to produce a different answer, we instead attempt to remove $\langle s_{19}\rangle$. If this succeeds, we then examine the removal of $P_{[17,18]}$ in the next iteration. The above process repeats until no further sentences can be removed (Lines 8–16). As with `Sequential_Greedy`, we repeatedly apply `Exponential_Greedy` to the provenance returned the previous iteration until the provenance no longer changes (Lines 23–26).

During this process, the opportunity to leverage prefix caching for cost optimization is maximized, as the prompt evaluated in the current iteration shares the longest possible prefix with the previous one, except for the sentences that have been examined and retained so far.

3.3 Quality and Cost Analysis

3.3.1 Quality Analysis. Let $\mathcal{S}_{prune} = \{$`Embedding-bottom-up`, `Embedding-top-down`, `LLM-bottom-up`, `LLM-top-down`$\}$ and $\mathcal{S}_{refine} = \{$`Sequential-Greedy`, `Exponential-Greedy`$\}$ be the set of strategies in the pruning and refinement phases, respectively.

Theorem 2. $\forall S \in \mathcal{S}_{refine}$, for all tasks $\langle T, Q, L\rangle$, and provenance $P \sqsubseteq T$, $S(P, \langle T, Q, L\rangle)$ always returns a minimal provenance.

We show the formal proof of Theorem 2 below. To return a minimal provenance, BLIP does not make any assumptions about the task (e.g., weak or strong monotonicity). These assumptions are only necessary if a strictly-minimal provenance is needed (Theorem 1).

Proof. We prove Theorem 2 by contradiction. Given a provenance P for the answer A, let the provenance returned by S be P_S, where $S \in \mathcal{S}_{\text{refine}}$. If P_S is not minimal, there must exist a sentence $s \in P_S$ such that $P_S \setminus s$ produces an answer equivalent to A, i.e., $I(L(P_S \setminus s), A)$ is *True*. If $S =$ `Sequential_Greedy`, this contradicts the algorithm's behavior (Lines 3 and 17–21 in Algorithm 2), since `Sequential_Greedy` is applied in multiple passes to the provenance returned from the previous iteration, ensuring that removing any sentence from it results in an answer that is not equivalent to A. A similar contradiction can be established for $S =$ `Exponential_Greedy`. Thus, P_S is a minimal provenance returned by any strategy in $\mathcal{S}_{\text{refine}}$. □

Theorem 3. $\forall S_i \in \mathcal{S}_{prune}$ and $S_j \in \mathcal{S}_{refine}$, for any task $\langle T, Q, L\rangle$, let $P = S_i(\langle T, Q, L\rangle)$ be the provenance returned by S_i. $S_j(P, \langle T, Q, L\rangle)$ always returns a minimal provenance for $\langle T, Q, L\rangle$.

The correctness of Theorem 3 follows directly from the fact that any strategy S_i in $\mathcal{S}_{prune}$ returns a provenance. Although heuristics are explored in rankers (e.g., embedding-based or LLM-based) and enumerations (e.g., bottom-up or top-down) to reduce cost and latency, Theorem 3 guarantees that BLIP returns a minimal provenance for any data processing task, including non-monotonic tasks. We provide below an example of how BLIP behaves on a non-monotonic task to illustrate this guarantee. Consider a non-monotonic task where $T = \langle s_1, s_2, s_3, s_4\rangle$, and only $\langle s_1\rangle$ and $\langle s_1, s_2, s_3\rangle$ are verifiable provenances among all true subsets of T (T itself is a provenance by definition). In this case, $\langle s_1\rangle$ is a strictly-minimal provenance, while both $\langle s_1\rangle$ and $\langle s_1, s_2, s_3\rangle$ are minimal provenances. BLIP returns $\langle s_1, s_2, s_3\rangle$, and is not able to return $\langle s_1\rangle$ since any superset of it formed by adding one more sentence (e.g., $\langle s_1, s_2\rangle$) is not a provenance. For the above task, if $\langle s_1, s_2\rangle$ is also a provenance, then it becomes a weakly-monotonic task. BLIP then returns $\langle s_1\rangle$

by searching along the path from $\langle s_1, s_2, s_3\rangle$, to $\langle s_1, s_2\rangle$, and finally $\langle s_1\rangle$, using Algorithm 2 (Line 17-26).

3.3.2 *Cost of Bottom-up versus Top-down.*

Consider the `bottom_up` and `top_down` strategies, with the text T as a list of m equal-sized blocks ranked by embeddings or LLMs. We focus on embeddings, but the analysis is similar for any ranker. Let C_{BU} and C_{TD} denote the total cost of `embedding_bottom_up` and `embedding_top_down`, respectively. Assume that the minimal provenance is contained within the top-k out of m total blocks. When there are multiple minimal provenances, we consider k to be the smallest number among them.

To determine which strategy incurs a lower cost given k, we analyze how the cost ratio $\frac{C_{BU}}{C_{TD}}$ varies with respect to k. To this end, we approximate the cost by counting the total number of block accesses in LLM verification calls, i.e., invocations of $I(L(P, Q), A)$,where a block is counted multiple times if it appears in multiple LLM calls. We also assume that the LLM is accurate in provenance verification. Specifically, if a subsequence T' is a superset of any minimal provenance mp (i.e., $mp \subseteq T'$), then the verification function returns true: $I(L(T', Q), A) = \text{True}$. We will later relax this assumption and analyze its impact on our results. Note that under this assumption, both `bottom_up` and `top_down` return exactly the top-k blocks as the provenance in this phase.

The cost of `bottom_up` is computed as: $C_{BU} = \sum_{i=1}^{k} i = \frac{k(k+1)}{2}$.

We use a recursive expression to compute the cost of `top_down`. Let $C(l, r)$ be the total number of blocks used in the iteration where `top_down` is searching the blocks in the interval $[l, r]$ in Algorithm 1. In this case, $C_{TD} = C(1, m)$.

$$C(l,r) = \begin{cases} 0, & l = r, \\ \text{mid}(l,r) + C(l,\ \text{mid}(l,r)-1), & k \le \text{mid}(l,r), \\ \text{mid}(l,r) + C(\text{mid}(l,r)+1,\ r), & k > \text{mid}(l,r). \end{cases}$$

where $\text{mid}(l,r) = \left\lceil (l+r)/2 \right\rceil$. In this expression, the number of blocks considered in each iteration, i.e., $\text{mid}(l, r)$, is accumulated until the binary search terminates at $l = r$.

We present a breakdown of the cost comparison based on different values of k relative to m, the total number of blocks.

Theorem 4. When $k \le L_m$, where $L_m = \frac{\sqrt{8m-7}-1}{2}$, we have $C_{BU} \le C_{TD}$; when $k \ge U_m$, where $U_m = \frac{-1+\sqrt{1+8(mlog_2 m - m + 1)}}{2}$, we have $C_{BU} \ge C_{TD}$. For $L_m < k < U_m$, one must evaluate C_{BU} and C_{TD} explicitly to determine which is larger.

We show the detailed proof below. As implied in Theorem 4, bottom_up is preferred when the provenance size is relatively small (i.e., k is smaller than L_m). In contrast, top-down wins when k is large (greater than U_m).

Proof of Theorem 4. During binary search in the top_down approach, consider the lower and upper bounds of C_{TD}. In the best case, in every iteration it moves to the left half interval, leading to a cost $C_{TD}^L = \lceil \frac{m}{2} \rceil + \lceil \frac{m}{4} \rceil + \cdots + 1 \ge m - 1$. In the worst case, in every iteration it moves to the right half interval. In this case, the first iteration incurs a cost of $\lceil \frac{m}{2} \rceil$, and the second iteration incurs a cost of $\lceil \frac{m}{2} \rceil + \lceil \frac{m}{4} \rceil$. Thus, the i-th iteration has a cost of $C_i = \sum_{j=1}^{i} \lceil \frac{m}{2^j} \rceil$. The total cost is $C_{TD}^U = \sum_{i=1}^{h} C_i$, where $h = \lceil \log_2 m \rceil$. We have,

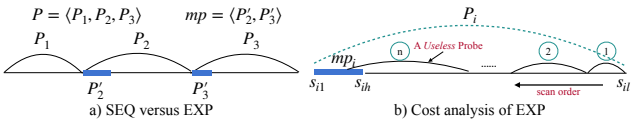


Figure 5: `SEQ` versus `EXP`. P refers to the provenance returned from the first phase, and $mp \sqsubseteq P$ refers to a minimal provenance.

$$C_{TD}^U = mlog_2 m - m + 1 \tag{3}$$

Solving $C_{BU} \le C_{TD}^L$ leads to,

$$k \le \frac{\sqrt{8m-7}-1}{2} \tag{4}$$

Soving $C_{BU} \ge C_{TD}^U$ gives,

$$k \ge \frac{-1+\sqrt{1+8(mlog_2 m - m + 1)}}{2} \tag{5}$$

We denote by $U_m = \frac{-1+\sqrt{1+8(mlog_2 m-m+1)}}{2}$ and $L_m = \frac{\sqrt{8m-7}-1}{2}$, respectively. When $k \le L_m$, we have $C_{BU} \le C_{TD}$. When $k \ge U_m$, we have $C_{BU} \ge C_{TD}$. For $L_m < k < U_m$, one must evaluate the two formulas explicitly to determine which is larger. □

3.3.3 *Cost of Sequential_Greedy versus Exponential_Greedy.*

Taking the provenance, denoted as P, returned by the *prune* phase as input, both Sequential_Greedy (`SEQ` for short) and Exponential_Greedy (`EXP` for short) return a minimal provenance, denoted as $mp \sqsubseteq P$. Let C_{SEQ} and C_{EXP} be the cost of `Sequential_Greedy` and `Exponential_Greedy`, when applied to P. To analyze C_{SEQ} and C_{EXP}, we assume both strategies return the same minimal provenancemp, as illustrated in Figure 5-a, where the blue intervals represent mp.

Irrespective of the distribution of the minimal provenance mp within P, any input sequence P can be decomposed into a list of subsequences (e.g., $\langle P_1, P_2, P_3\rangle$), where the first (possibly empty) subsequence P_1 has no contribution to mp, followed by P_2, P_3, ..., each of which has a prefix P'_2, P'_3 that is part of mp. This decomposition is useful for comparing `EXP` versus `SEQ` across different types of sequences. For P_1, `EXP` is clearly preferred over `SEQ`, as `EXP` eliminates sentences in logarithmic time, whereas `SEQ` does so linearly.

We now focus on the cost comparison for sequences such as P_2 or P_3. W.L.O.G., consider such a sequence $P_i = \langle s_{i1}, s_{i2}, \ldots, s_{il}\rangle$, where the leading subsequence $mp_i = \langle s_{i1}, \ldots, s_{ih}\rangle$ is part of the minimal provenance mp, as illustrated in Figure 5-b. Similar to the cost analysis in Section 3.3.2, we approximate the cost by counting the total number of block accesses in LLM verification calls and assume the LLM is accurate in provenance verification. `EXP` performs logarithmic deletion over the subsequence $P_i \setminus mp_i$ and linear deletion over mp_i, whereas `SEQ` always performs linear deletion over the entire P_i, both by scanning sentences in decreasing order of their indexes. However, when an LLM verification fails, `EXP` resets the probing window step back to 1, rendering the current LLM call ineffective in deleting any sentences, thus, a *useless* probe. To analyze the cost comparison between `EXP` and `SEQ`, we need to account for both the savings from `EXP`'s logarithmic deletion and the additional cost incurred by useless probes.

Theorem 5. Given a sequence $P_i = \langle s_{i1}, s_{i2}, \ldots, s_{il}\rangle$ with a leading subsequence $mp_i = \langle s_{i1}, \ldots, s_{ih}\rangle$ that is part of the provenance,

let $g = l - h$, we have:

$$C_{SEQ} = \Theta(gl + h^2),$$
$$\Omega(g + h^2) \leq C_{EXP} \leq O(g \log_2^2 g + h \log_2^2 g + h^2)$$

The cost of C_{SEQ} and C_{EXP} are broken down as follows,

$$g = \omega(1) \implies C_{EXP} = o(C_{SEQ}),$$
$$g = \Theta(1) \implies C_{EXP} = \Theta(C_{SEQ}).$$

We show the proof below. As shown in Theorem 5, $C_{\texttt{EXP}}$ is asymptotically no worse and often cheaper than $C_{\texttt{SEQ}}$ in most cases. In practice, since the provenance size often remains constant relative to the sequence length, i.e., $r = \omega(1)$, `EXP` is cheaper than `SEQ` when l and r become larger, implied by $C_{EXP} = o(C_{SEQ})$.

Proof. First, consider computing C_{SEQ}. Let $g := l - h$ be the number of sentences that are not part of the provenance. To linearly probe each sentence in $P \setminus mp$ using `SEQ`, the number of data blocks per LLM verification is $l - 1, l - 2, \ldots, l - g$, leading to a total cost of $gl - \frac{g(g+1)}{2}$. For probing each sentence in mp, since no sentence can be removed, `SEQ` incurs an additional cost of $h(h-1)$, resulting in the total cost:

$$C_{SEQ} = gl - \frac{g(g+1)}{2} + h(h-1).$$

We have the tight bound of C_{SEQ} as,

$$\boxed{C_{\texttt{SEQ}} = \Theta(gl + h^2)}$$

since $gl \geq g$ and $h^2 \geq h(h-1)$, while $\frac{1}{2}g(g+1) \leq g^2 \leq gl + g^2 \leq 2gl$.

Now consider C_{EXP}. To probe and delete sentences that are not part of the provenance, `EXP` performs k successful probes followed by one failure, then resets the window size to 1 and recursively repeats the same probing process. The cost in this stage is broken down into the cost of successful probes and the failure probe. The former is computed as $\sum_{i=0}^{k-1}(h + g - 2^i) = k(h+g) - 2^k + 1$, where $k = \lceil log_2 g \rceil$, the number of successful probes.

$$\underbrace{k(h+g) - 2^k + 1}_{\text{successful probes}} + \underbrace{\max\{0, h + g - 2^{k+1}\}}_{\text{one failure}} = O((h+g)\log_2 g).$$

After a failure probe, at most $2^k - 1 < \frac{1}{2}g$ non-provenance sentences remain, so the recursion depth is at most $\lceil \log_2 g \rceil$. Thus, we have the upper asymptotic bound of $C_{\texttt{EXP}}$ as follows, where $O(h^2)$ accounts for the cost of probing the h provenance sentences.

$$\boxed{C_{\texttt{EXP}} = O(glog_2^2 g + h \log_2^2 g + h^2)}.$$

Now we develop the lower asymptotic bound of C_{EXP} as below,

$$\boxed{C_{\texttt{EXP}} = \Omega(g + h^2)}.$$

Here, every non-provenance sentence in $P \setminus mp$ contributes a cost of at least one block to the overall cost.

Comparing the two strategies, we have the tight bound $C_{SEQ}(l, h) = \Theta(gl + h^2)$ with $g = l - h$, while `EXP` satisfies $\Omega(g + h^2) \leq C_{EXP}(l, h) \leq O(g \log_2^2 g + h \log_2^2 g + h^2)$. Because gl grows strictly faster than $g \log g$ once $g = \omega(1)$, we obtain $C_{EXP} = o(C_{SEQ})$ for sequence containing more than a constant number of non-provenance sentences; conversely, when $g = \Theta(1)$ both costs are the same order. Overall, we have,

$$\boxed{\begin{array}{ll} g = \omega(1) & \implies C_{EXP} = o(C_{SEQ}), \\ g = \Theta(1) & \implies C_{SEQ} = \Theta(C_{EXP}). \end{array}}$$

□

3.3.4 Recommended Approach. We have presented two different pruning (bottom-up and top-down) and refinement strategies (SEQ and EXP). Theorems 4 and 5 (as well as our empirical study in Section 5.2) show that the best pruning and refinement approach depends on the size of the provenance, quality of ranking, as well as how provenance is distributed within T. Given absence of knowledge about these factors, we provide the following recommendations to decide which pruning and refinement strategy to use.

Recommended Pruning strategy. We propose an adaptive strategy for pruning. Theorem 4 showed that if provenance is the first L_m blocks, top-down performs best while if it is in the last U_m blocks, then the bottom-up performs best. We use this insight to design the following adaptive strategy. We use $CP = \frac{L_m + U_m}{2}$ to approximate a *crossover point*. Given a ranking approach, the adaptive strategy initially applies bottom-up to probe the ranked blocks. If the top-CP blocks fail to produce the correct answer, the strategy switches to top-down to complete the search. Our empirical results (Section 5.2) show that this simple strategy often outperforms both methods.

Recommended Refinement strategy. SEQ performs best when a short provenance can be found at the beginning of the document (i.e., h is small in Theorem 5) while EXP performs better when documents are longer. This is consistent with our empirical observation: we recommend using SEQ if the number of sentences in the provenance returned from the pruning stage is below a threshold t ($t = 10$ reported in Section 5.2), but to use EXP otherwise.

3.3.5 BLIP Scope. We so far have shown that BLIP returns a minimal provenance for any LLM-powered data processing [13, 32, 47, 65, 67] task, as discussed in Section 3.3.1; and for any SPJRU (i.e., $\mathcal{RA}^+$) task, the provenance is well-defined and useful, as discussed in Section 2.5. However, whether BLIP can do so efficiently depends on the task itself as well as the abilities of the ranking method used.

First, if the provenance for the task is large, then the cost of BLIP increases, as discussed in Theorem 5—guaranteeing minimality of a large set is difficult as any sentence can be redundant and our algorithm must check if the removal of any sentence can lead to a smaller provenance. Second, the quality of the ranking method affects the cost—the better the ranker, the more effectively BLIP can prune out blocks that are not part of the provenance. The quality of the ranker depends on the capabilities of the embedding model (or the LLM used for ranking), as well as task and data characteristics. Embedding models are commonly used for semantic retrieval tasks both for text [68] and tabular data [34, 48], and as we see in our experiments, can provide accurate ranking for common LLM-powered text and table processing tasks done by LLMs. However, embedding models may not be well-suited for ranking in tasks that require logical operations (e.g., when we have a predicate `A = 4`, and all values of `A` are between 3 to 5). Such queries should be performed using SQL, e.g., by using an LLM to rewrite the query Q using text-to-SQL as opposed to the LLM processing data directly. In this case,

Algorithm 3: `top-k-provenance`

Input: Q, T, L, A, F, I, k

1 $T = \langle b_1, \ldots, b_m \rangle$
2 $H \leftarrow$ empty max heap, $P \leftarrow$ empty list
3 $H.append((T_{1,m}, 1))$
4 **while** $H \neq \emptyset$ **do**
5 $(T_{l,r}, score_{l,r}) \leftarrow H.pop()$
6 **if** $l < r$ **then**
7 $mid = \lceil \frac{l+r}{2} \rceil$
8 $score_{l,mid} \leftarrow F(L(T_{l,mid}, Q), A)$
9 $score_{mid+1,r} \leftarrow F(L(T_{mid+1,r}, Q), A)$
10 $c_1 \leftarrow (l == r) \wedge (score_{l,r} \geq 0.5)$
11 $c_2 \leftarrow (l < r) \wedge (score_{l,r} \geq 0.5) \wedge (score_{l,mid} < 0.5) \wedge (score_{mid+1,r} < 0.5)$
12 **if** $(c_1 \vee c_2) == True$ **then**
13 **if** $c_1 == True$ **then**
14 $P_{cur} =$ `refine`$(T_{l,r})$
15 **else**
16 $P_{cur} =$ `refine(prune(`$(T_{l,r})$`))`
17 **if** `user_verify`(P_{cur}) *== Positive* **then**
18 **Return** P_{cur}, True
19 $P \leftarrow P \cup P_{cur}$
20 **if** $c_2 == False$ **then**
21 $H.append((T_{l,mid}, score_{l,mid}))$
22 $H.append((T_{mid+1,r}, score_{mid+1,r}))$
23 **if** $|P| \geq k$ **then**
24 **Return** P

existing provenance tracking solutions for databases [11, 17, 26] are more effective. We leave how our approach can be combined with query answering with SQL to the future work.

Note that adopting `Exponential_Greedy` improves the worst-case performance of the two-phase approach when the provenance returned from the first phase is large, particularly when the provenance distribution is local. This is because it offers a logarithmic-factor reduction in pruning sentences that do not contribute to the answer, thereby making the approach more robust.

4 FINDING TOP-K PROVENANCE

The two-phase approach described in Section 3 returns a single minimal provenance. However, it may not be sufficient for users to verify the correctness of an answer. For example, consider a question asking for the publication year of a scientific paper in Figure 7. Provenances 1 and 2 represent two distinct provenances that support the correctness of the answer. In contrast, Provenance 3 happens to return the same answer without supporting its correctness, as it refers a different paper citation in the *Reference* section.

A provenance is considered *positive* (e.g., Provenances 1 and 2 in Figure 7) if it supports the correctness of the answer. Otherwise, it is *negative* (e.g., Provenance 3). The distinction between positive and negative provenances can ultimately only be determined by a human than an LLM, as both types of provenance pass the LLM's verification process (i.e., they both reproduce the same answer derived from the full text). To address this issue and to enable users to explore multiple distinct provenances and gain trust in the correctness of the answer, we develop an approach, outlined in Algorithm 3, to return the k minimal provenances. During this process, if users identify any one of the provenances as positive, then the algorithm terminates.

Algorithm. We illustrate Algorithm 3 for inferring k provenances using a running example shown in Figure 7. We let the text T be a list of m equal-sized blocks, and we aim to find three minimal provenances ($k = 3$).

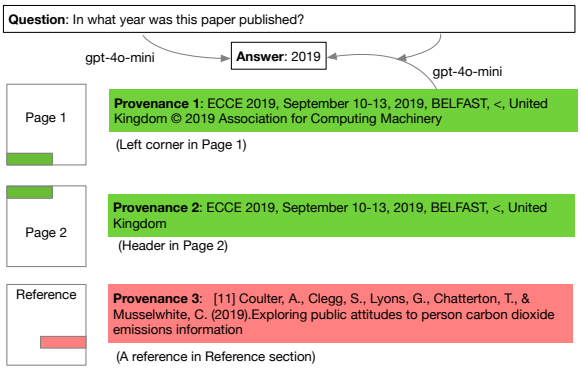


Figure 6: Top-k Provenance.

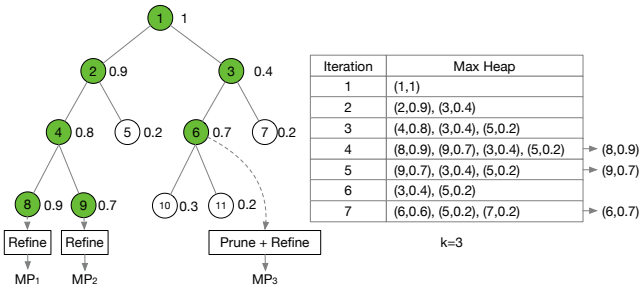
Initially, we start from T (or equivalently, $T_{1,m}$, denoting the sequence of blocks from B_1 to B_m), represented by the root node 1. In the second iteration, we split $T_{1,m}$ into left and right halves, corresponding to its child node 2 $T_{1,\lceil m/2 \rceil}$ and node 3 $T_{\lceil m/2 \rceil+1,m}$, respectively. For each node, we compute a score as the likelihood that the corresponding block can reproduce A. Such a score is 1 if the answer A' produced by the block is lexically identical to A; otherwise, we leverage LLMs to generate the score using the prompt `Top-k-Eval-Prompt` below. For example, the scores of nodes 1 and 2 are 1 and 0.9.

`Top-k-Eval-Prompt`: [Question], [Answer 1], and [Answer 2] are placeholders for the question Q, the answer $L(T_{l,\text{mid}}, Q)$ evaluated on $T_{l,\text{mid}}$, and A, respectively, in Line 8-9 in Algorithm 3. Note that when value is *False*, we use 1− score as its likelihood.

```
Top-k-Eval-Prompt: Given the following question, [Question], and two answers, [Answer 1] and [Answer 2]. Determine whether the two answers are equivalent in meaning. Return the result as a JSON object with the following format: value: true if the answers are equivalent, false otherwise. score: a real number between 0 and 1 representing the likelihood that your judgment is correct.
```

We create a max heap H that stores pairs consisting of a text block and its corresponding score. In each iteration, we expand the node with the highest likelihood score from the max heap and remove the node that is already expanded. For example, node 1 is removed from H in the second iteration, and node 2 is expanded in the next iteration since it has the highest score. We then insert its two children (node 4 and 5)—obtained by splitting the current text sequence into left and right halves—into the max heap and rank them based on their likelihood scores.

The above node splitting continues recursively until either of the following two conditions is met. First, when we reach a node that contains only one block (e.g., node 8) and it passes verification (i.e., reproduces the answer), we return it by invoking `refine` to obtain the corresponding minimal provenance. In this example, node 8 is the first provenance returned by the `top-k-provenance` algorithm, assuming it is distinct from the provenance returned by the two-phase approach. Second, when we reach an internal node (e.g., node 6) whose score is no less than 0.5, but both of its child nodes have scores lower than 0.5 (e.g., nodes 10 and 11), node 6 is returned as



Iteration	Max Heap
1	(1,1)
2	(2,0.9), (3,0.4)
3	(4,0.8), (3,0.4), (5,0.2)
4	(8,0.9), (9,0.7), (3,0.4), (5,0.2)
5	(9,0.7), (3,0.4), (5,0.2)
6	(3,0.4), (5,0.2)
7	(6,0.6), (5,0.2), (7,0.2)

Figure 7: Top-k Provenance.

	Question Types				# of Hops		
	Look-up	Aggregation	Reasoning	Judge	1-Hop	2-Hops	≥3-Hops
Qasper	54.1%	4%	24.4%	17.6%	72.3%	22.2%	5.6%
NL_DEV	75.2%	4%	18%	2.8%	42.3%	52.1%	5.6%
HotpotQA	68.1%	1.8%	22.8%	7.4%	7.6%	57.3%	35.1%
CUAD	17.1%	6.3%	5.2%	71.4%	7.3%	58.5%	34.1%
PubMedQA	6%	2%	51%	41%	9.6%	56.2%	34.2%

Table 5: Characteristics of Questions across Workloads.

the third provenance. This provenance is then further refined using the two-phase approach to obtain a minimal provenance.

Discussion. The `top-k-provenance` algorithm does not guarantee returning all distinct minimal provenances, but only a subset of them in T. We argue that this is often sufficient in many scenarios. If the question over T is correctly answered by LLM L, then finding a single positive provenance suffices for user trust, and the algorithm can terminate without exploring all minimal provenances. On the other hand, if the answer returned by L on T is incorrect, ideally all minimal provenances that are negative are expected to trust that answer is wrong.

5 EVALUATION

We evaluate the performance ofBLIP through experiments conducted on five real-world, two synthetic workloads, as well as a user study.

5.1 Evaluation Setup

Datasets & Workloads. We conduct experiments on five commonly used question-answering workloads: *Qasper [20]*, *NL_DEV* [38], *HotpotQA [63]*, *CUAD* [30], and *PubMedQA* [35], covering the domains of scientific papers, Wikipedia, legal contracts, and medical records. For each workload, we sampled 500 distinct question-document pairs, ensuring that each question and document in a pair is unique. The average tokens per document is 8,972, 7,957, 1,233, 9,964, and 891 for Qasper, NL_DEV, HotpotQA, CUAD, and PubMedQA, respectively. Questions with diverse types and complexities are evaluated: about 72% require at least two reasoning hops, as shown in Table 5; Here, Look-up and Aggregation denote factual questions, Judge corresponds to boolean questions (i.e., yes/no), and Reasoning questions ask why or how, typically requiring longer explanatory answers. We additionally include two synthetic workloads, *Movie* and *Restaurant*, from TQA-bench [61], which focus on question answering over tables (TableQA), each containing 64,000 tokens and 100 questions.

True provenance is available for TableQA workloads but not for others. Since the latter five workloads are on question answering over plain text, true provenance is unknown, and even human annotators may disagree on the ground truth; there may also be multiple true provenances. For the TableQA workloads, Movie and Restaurant, true provenance (i.e., the set of tuples) is provided for question-answer pairs. We focus on multi-hop reasoning questions in TableQA workloads, which are common analytical queries over tables. Out of 100 questions per TableQA workload, half are single-hop reasoning questions, while the others involve 2 to 3 hops.

Compared Strategies. We first compare the five strategies in the pruning phase, along with ten combined two-phase strategies. `Sequential-Greedy` and `Exponential-Greedy` are abbreviated as `SEQ` and `EXP`, respectively. So `LLM_bottom_up_EXP` represents a strategy that uses `LLM_bottom_up` and `Exponential-Greedy` in the pruning and refinement phase, respectively. We also compare BLIP against a retrieval-based approach, RAG, using various context sizes (1%, 5%, and 10% of the full text), and against LLM-generated provenance as mentioned in Section 1.

Implementation. When performing `prune` in Algorithm 1, we divide the text into $m = 20$ equal-sized blocks. The embedding- and LLM-based rankers used to sort the blocks are implemented with the open-source embedding model `all-mpnet-base-v2` and LLM `Mistral 7B`. Both embeddings and relevance scores are computed offlinefor each document-question pair. For question answering and provenance inference, we use `gpt-4o-mini` and `gemini-2-flash`. Results reported here are based on `gpt-4o-mini`. `Gemini-2-flash` has similar findings, in Table 10.

Metric. We first report the *accuracy*, defined in Section 1, as the fraction of document–question pairs for which the inferred provenance can reproduce an answer equivalent to the one from the full text. We further report the *size ratio*, defined as the number of tokens in the generated provenance divided by that in the full text. We additionally report the *latency* and the *cost ratio* that is defined as the cost of finding the provenance divided by the cost of question answering. For strategies using an embedding-based ranker, we exclude the latency for building embeddings and report only the latency for provenance inference, as embeddings can be computed offline. Finally, observing that an incorrect provenance requires humans to read the entire document, we report *human review effort*, measured as the fraction of the document corresponding to the provenance (i.e., size ratio) if accurate (i.e., the provenance is verifiable), and as one plus the size ratio otherwise.

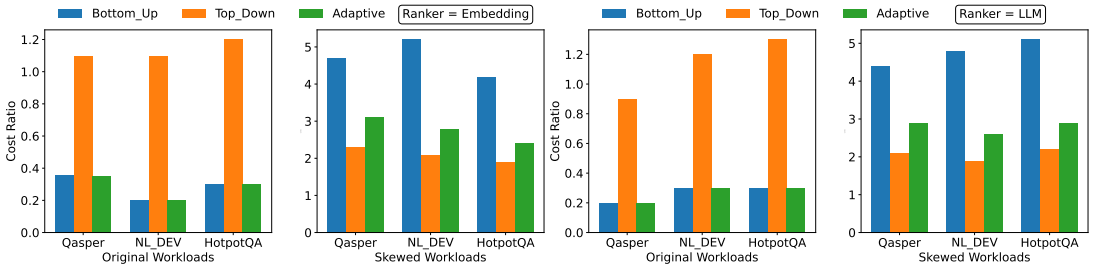
We also measure whether methods can recover the true provenance P_{true} for TableQA workloads by introducing two metrics: recovery (R) and exact-recovery (eR). Provenance is represented as a set of tuples in the tables. R is defined as the percentage of questions where a predicted provenance P_{pre} contains P_{true} over all questions, i.e., $R = \frac{|P_{true} \subseteq P_{pre}|}{|QA|}$, while eR requires P_{pre} to be an exact match with P_{true}, i.e., $eR = \frac{|P_{true} = P_{pre}|}{|QA|}$. We note that the *accuracy* defined measures how often a provenance inferred by BLIP can reproduce the answer (i.e., is verifiable), while *recovery* measures whether the inferred provenance matches the true provenance.

5.2 Experiments

Experiment 1: Cost, Latency, and Provenance Size of BLIP. We present the cost ratio, latency, and the size ratio of the provenance generated by BLIP in Table 6. We make several observations. First, strategies in the pruning phase effectively eliminate text portions unrelated to answers, with adaptive strategies reducing average provenance size ratios to 10.2%, 12.8%, and 20.2% for Qasper,

Type	Strategy	Qasper (8972 Tokens)			NL_DEV (7957 Tokens)			HotpotQA (1233 Tokens)		
		Size Ratio (%)	Cost Ratio	Latency	Size Ratio (%)	Cost Ratio	Latency	Size Ratio (%)	Cost Ratio	Latency
Pruning Strategies	LLM_bottom_up	9.7	**0.2**	**2**	13.6	0.3	2	19.8	**0.3**	1.2
	LLM_top_down	12.6	0.9	3	15.3	1.2	2.9	22.3	1.3	1.9
	LLM_adaptive	9.8	**0.2**	2.1	13.5	0.3	2	20	**0.3**	**1.1**
	Embedding_bottom_up	10.5	0.36	2.1	12.2	**0.2**	**1.9**	20.2	**0.3**	1.3
	Embedding_top_down	12.4	1.1	3.1	14.8	1.1	2.6	23.2	1.2	2
	Embedding_adaptive	10.6	0.35	2.1	12.1	**0.2**	**1.9**	20.3	**0.3**	1.2
Two-Phase Strategy: Pruning Strategies + Refinement Strategies	LLM_bottom_up_EXP	3.8	**1.2**	8.4	4.8	**1.1**	**8.5**	7.7	**1.2**	**3.5**
	LLM_bottom_up_SEQ	3.6	**1.3**	9.7	4.9	1.3	12.2	7.8	1.5	3.8
	LLM_top_down_EXP	3.7	1.7	11.2	5.1	1.9	10.5	8.1	2.2	4.1
	LLM_top_down_SEQ	4.1	2.1	13.1	4.9	2.4	14.2	8	2.9	4.7
	LLM_adaptive_EXP	3.7	**1.2**	**8.1**	4.8	**1.1**	**8.5**	7.7	**1.1**	**3.3**
	LLM_adaptive_SEQ	3.6	**1.3**	9.3	4.8	**1.2**	11.4	7.8	1.4	**3.7**
	Embedding_bottom_up_EXP	3.4	**1.3**	**8.2**	4.6	**1.1**	**8.1**	7.9	**1.2**	4.1
	Embedding_bottom_up_SEQ	3.4	1.4	11.3	4.7	**1.2**	9.9	7.8	1.4	4.5
	Embedding_top_down_EXP	4.3	1.9	12.9	5.2	2.2	10.8	8.2	2.4	5.8
	Embedding_top_down_SEQ	4.1	2.4	15.2	4.8	2.7	13.5	7.8	2.9	6.4
	Embedding_adaptive_EXP	3.3	**1.3**	**8**	4.6	**1.1**	**8.3**	7.9	**1.1**	4
	Embedding_adaptive_SEQ	3.4	1.4	10.8	4.7	**1.2**	9.6	7.8	**1.3**	4.2

Table 6: Size Ratio, Cost Ratio, and Latency of Strategies in `Qasper`, `NL_DEV`, and `HotpotQA` Workloads. The top-1 pruning strategies, and the top-3 two-phase strategies are highlighted in bold for both cost ratio and latency. `EXP` and `SEQ` stand for `Exponential-Greedy` and `Sequential-Greedy`.



(a) Ranker = Embedding (b) Ranker = LLM

Figure 8: Cost ratios of pruning strategies on three real-world workloads and their skewed versions.

Method	Configuration	Qasper	NL_DEV	HotpotQA	CUAD	PubMedQA
LLM	**gpt-4o-mini**	0.4	0.37	0.46	0.42	0.39
	gpt-4o	0.36	0.26	0.3	0.31	0.34
	gemini-2-flash	0.6	0.65	0.47	0.47	0.42
Retrieval	**RAG-1%**	0.82	0.84	0.89	0.78	0.92
	RAG-5%	0.59	0.64	0.7	0.66	0.81
	RAG-10%	0.32	0.41	0.54	0.48	0.61
BLIP	**E_adaptive_EXP**	**0.033**	**0.046**	**0.079**	**0.03**	**0.12**

Table 7: Human Review Effort. (E stands for Embedding in BLIP.)

Dataset	Size Ratio(%)	Cost Ratio	Latency
CUAD	2.8	1.4	9.8
PubMedQA	11.3	1.2	4.2

Table 8: Evaluation Results of BLIP (the overall recommended strategy, Embedding_adaptive_EXP) on CUAD and PubMedQA.

Dataset	Ranker	$\mathcal{S}_{Prune}$	Bottom-up vs. Top-down			EXP vs. SEQ
			CP_1	Q1	Q3	CP_2 (Pct)
Qasper	LLM	BU	8.2	1	2	8.4 (35.2%)
		TD	8.2	1	2	9.3 (32.7%)
	Embedding	BU	7.4	1	3	8.9 (31.9%)
		TD	7.4	1	3	9.5 (28.6%)
NL_DEV	LLM	BU	6.9	1	3	8.5 (38.4%)
		TD	6.9	1	3	9.1 (35.2%)
	Embedding	BU	6.7	1	2	8.2 (40.1%)
		TD	6.7	1	2	8.9 (38.5%)
HotpotQA	LLM	BU	7.2	1	3	7.5 (40.2%)
		TD	7.2	1	3	8.1 (38.4%)
	Embedding	BU	7.4	1	2	7.2 (38.1%)
		TD	7.4	1	2	8.9 (37.5%)

Table 9: Crossover Points (CP_1 and CP_2). BU and TD stand for bottom_up and top_down, respectively. $Q1$ and $Q3$ are the first and third quartiles of the returned data blocks when comparing bottom-up and top-down. Pct stands for the percentile of cases where the number of input sentences is less than CP_2.

NL_DEV, and HotpotQA, respectively. These strategies incur only 0.28×, 0.25×, and 0.3× the cost of answering questions over the full text, with latencies averaging 2.1, 1.9, and 1.1 seconds. Among them, `LLM_adaptive` ***reduces provenance size to around 9.8% of the full text, achieving a 0.2× cost ratio and an average latency of 2.1 seconds*** across the workloads.

Strategies in $\mathcal{S}_{refine}$ further reduce the provenance size ratio to about ***3.7%, 4.7%, and 7.7% on Qasper, NL_DEV, and HotpotQA***, respectively. Here, `Exponential_Greedy` (i.e., strategies ending with `EXP`) have up to a ***1.1× cost ratio and an average latency of 8.1 seconds*** across workloads, demonstrating that two-phase strategies produce minimal provenances at only slightly higher cost than full-text QA. Latency can be further reduced via parallelization. We have consistent observations on the CUAD and PubMedQA workloads, as shown in Table 8, where BLIP incurs a slightly higher cost (with a cost ratio of 1.3) than question answering over the entire input to infer a minimal provenance efficiently.

Experiment 2: Accuracy and Human Review Effort. We have already presented the accuracy for retrieval-based (RAG) and LLM-generated provenance in Table 1, where BLIP strategies guarantee **an accuracy of 1, over 30% higher than the best baseline** with a comparable provenance size (i.e., LLM-generated provenance using `gpt-4o`). Accuracy of 1 on non-monotonic tasks (~5% of queries; Table 3) is consistent with Theorem 2.

We report the human review effort in Table 7. Compared with RAG-based and LLM-generated provenance, BLIP requires substantially less human review effort—only ***3% to 12%*** across five workloads. This is because BLIP always returns a minimal verifiable provenance, both effective and efficient for human verification.

Experiment 3: Bottom_up, Top_down, and Adaptive strategies. Although bottom_up and adaptive strategies outperform their

Type	Strategy	Qasper (8972 Tokens)			NL_DEV (7957 Tokens)			HotpotQA (1233 Tokens)		
		Size Ratio (%)	Cost Ratio	Latency	Size Ratio (%)	Cost Ratio	Latency	Size Ratio (%)	Cost Ratio	Latency
Pruning Strategies	LLM_bottom_up	10.3	**0.3**	2	14.1	0.4	2.2	17.3	0.3	1.7
	LLM_top_down	13.4	0.8	2.7	14.7	1.4	3.2	19.5	1.1	2.2
	LLM_adaptive	10.2	**0.3**	**1.8**	13.9	0.3	**2.1**	17.7	**0.2**	**1.6**
	Embedding_bottom_up	9.4	0.5	1.9	13.1	**0.3**	2.4	18.7	0.4	1.8
	Embedding_top_down	10.8	1.2	2.8	14.5	1.2	3.1	20.5	1.2	2.5
	Embedding_adaptive	9.7	0.4	2	13.1	**0.3**	**2.2**	18.9	0.4	1.7
Two-Phase Strategy: Pruning Strategies + Refinement Strategies	LLM_bottom_up_EXP	4.1	**0.9**	**7.5**	5.2	**0.9**	**9.1**	7.3	**0.8**	**3.7**
	LLM_bottom_up_SEQ	3.9	**1.1**	9.4	5	**1.1**	11.9	7.2	1.1	3.9
	LLM_top_down_EXP	4	1.4	10.4	5.1	1.6	10.9	7.6	1.9	4
	LLM_top_down_SEQ	4.3	1.8	14.2	5.4	2.1	13.7	7.8	2.3	4.3
	LLM_adaptive_EXP	4	**0.9**	8.6	5.2	**1.1**	**8.9**	7.4	**0.8**	**3.5**
	LLM_adaptive_SEQ	3.9	**0.9**	8.8	4.9	**1**	12.9	7.4	**0.9**	**3.7**
	Embedding_bottom_up_EXP	3.6	1.3	**7.6**	5	**1**	**8.4**	7.3	**1**	3.9
	Embedding_bottom_up_SEQ	3.7	**1.1**	10.6	4.9	**1.1**	9.6	7.6	1.4	4.2
	Embedding_top_down_EXP	3.9	1.7	11.3	5.2	1.7	12.1	7.9	1.8	4.6
	Embedding_top_down_SEQ	4	2	13.9	5.3	2.2	14.7	7.8	2.3	5.7
	Embedding_adaptive_EXP	3.6	**1.1**	**8.2**	4.9	**0.9**	**8.2**	7.5	**0.9**	**3.6**
	Embedding_adaptive_SEQ	3.7	**1.2**	10.3	4.8	**1**	**9.1**	7.6	**1**	3.8

Table 10: Gemini-2-flash: Size Ratio, Cost Ratio, and Latency of Strategies in `Qasper`, `NL_DEV`, and `HotpotQA` Workloads. The top-1 pruning strategies, and the top-3 two-phase strategies are highlighted in bold for both cost ratio and latency. `EXP` and `SEQ` stand for `Exponential-Greedy` and `Sequential-Greedy`.

Methods	Models	Movie (64k Tokens)						Restaurant (64k Tokens)					
		Q_{simple}			$Q_{complex}$			Q_{simple}			$Q_{complex}$		
		eR	*R*	*A*	*eR*	*R*	*A*	*eR*	*R*	*A*	*eR*	*R*	*A*
BLIP	E_adaptive	**1**	**1**	**1**	**0.98**	**1**	**1**	**0.98**	**1**	**1**	**0.96**	**1**	**1**
LLM	gpt-4o-mini	0.62	0.76	0.74	0.58	0.75	0.73	0.72	0.84	0.84	0.73	0.81	0.79
	gpt-4o	0.64	0.89	0.86	0.62	0.83	0.82	0.74	0.9	0.88	0.79	0.86	0.86
	gemini-2-flash	0.83	0.91	0.9	0.67	0.8	0.76	0.7	0.86	0.83	0.73	0.79	0.78
Retrieval	RAG-EXACT	0.44	0.44	0.44	0.32	0.32	0.32	0.68	0.68	0.68	0.62	0.62	0.62
	RAG-1%	0	0.48	0.48	0	0.42	0.41	0	0.82	0.81	0	0.74	0.74
	RAG-5%	0	0.72	0.7	0	0.58	0.56	0	0.92	0.91	0	0.86	0.86
	RAG-10%	0	0.8	0.76	0	0.74	0.71	0	0.96	0.93	0	0.94	0.93

Table 11: Recovery and Accuracy in TableQA Workloads. (E stands for Embedding.)

top_down counterparts in Table 6, we provide a breakdown to understand why. Section 3.3.2 shows that the cost of these strategies depends on the number of returned data blocks, with a suggested crossover point of $CP \approx 8.4$ when the text is divided into 20 blocks. We create a skewed dataset by upsampling question-document pairs so that 80% of the pairs have more returned data blocks than CP. Figure 8 reports the cost ratios on both the original and skewed workloads. Top-down is over 2× cheaper on the skewed workloads, while adaptive strategies provide a middle ground in both settings. To understand the trend of cost relative to the number of returned data blocks, we further plot the cost ratio of `LLM_bottom_up`, `LLM_top_down`, and `LLM_adaptive` for various numbers of returned data blocks in Qasper in Figure 9. The cost of the bottom_up strategy grows much faster than that of top_down—it is lower when fewer than 8.2 data blocks are returned but higher beyond that. This is because top_down examines blocks exponentially, while bottom_up proceeds linearly, resulting in a roughly quadratic cost ratio relative to the number of returned data blocks. We also report the distribution of returned data blocks in Table 9, where the actual crossover points, denoted by CP_1, are presented when comparing bottom_up and top_down. The results show that most question-document pairs have a small number of returned blocks, making bottom_up more cost-effective on average, whereas top_down is preferred in the skewed workloads.

Overall, we recommend the adaptive strategy when provenance size is unknown, as it combines both strategies' strengths while avoiding worst-case costs.

Experiment 4: Exponential-Greedy (EXP) versus Sequential-Greedy (SEQ) Strategies. Consider `SEQ` and `EXP` in the refine phase in Table 6. `EXP` consistently outperforms `SEQ`, 11.6%, 12%, and 18.6% cheaper, and 1.3×, 1.34×, and 1.1× faster for Qasper, NL_DEV, and HotpotQA, respectively. When the size of the input provenance is larger, `EXP` shows greater improvement over `SEQ`, up to 24% cheaper and 1.49× faster. Figure 10 plots their cost ratios vs. input size on Qasper: `SEQ` is preferred below 10 sentences, while `EXP` scales better beyond that. Similar observations hold for other strategies and workloads in Table 9, where CP_2 denotes the crossover point based on the number of input sentences. Specifically, when the input contains fewer than CP_2 sentences, `SEQ` has a lower cost for more than half of the question-document pairs. An average percentile (Pct for CP_2) below 40%, implying that only less than 40% of inputs favor `SEQ`, explains why `EXP` has an overall lower cost. Overall, when the number of sentences is small (10 is an empirical crossover point), `SEQ` is recommended for its simplicity and lower cost. Otherwise, `EXP` is clearly preferred.

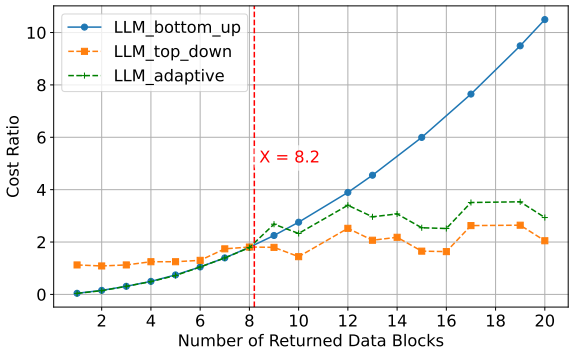


Figure 9: Cost Ratio of top_down and bottom_up on Qasper.

Experiment 5: Recovery of True Provenance in TableQA Workloads. To measure how well methods can recover the true provenance, we report both recovery (R) and exact recovery (eR) along with accuracy (A) for BLIP against four variants of RAG- and LLM-generated provenances in Table 11. BLIP uses an adaptive strategy using embedding model as the ranker, where `SEQ` or `EXP` is chosen based on the provenance size (fewer than 10 sentences or not) returned in the previous phase. All RAGs rank tuples based on embedding similarity to the question–answer pair, using the

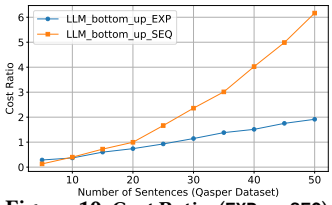


Figure 10: Cost Ratio. (EXP vs. SEQ)

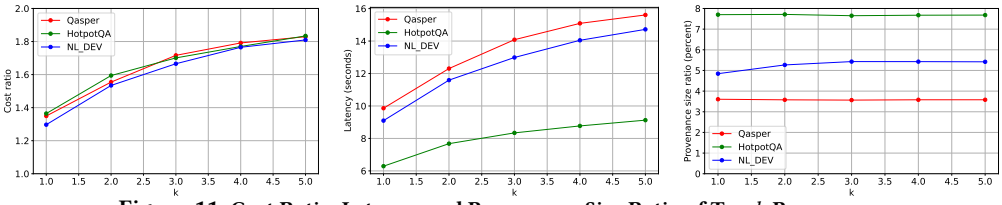


Figure 11: Cost Ratio, Latency, and Provenance Size Ratio of Top-k Provenance.

	police_reports		legal		paper		finance	
	H_q	H_e	H_q	H_e	H_q	H_e	H_q	H_e
BLIP	**4.1**	**75%**	**5**	**100%**	**4.9**	**88%**	**5**	**63%**
RAG	3.1	38%	3.1	25%	3.3	20%	2.8	0%

Table 12: User Study. (H_q ranges from 1 to 5 and measures the usefulness of the provenance, while H_e is the percentage of cases in which a provenance contains the right amount of information.)

same embedding model as BLIP. The top-k tuples are used in RAG-EXACT, where k matches the size of the true provenance. In the RAG-p% strategy, the top-p% of tuples are returned.

BLIP can always recover the superset of true provenance as indicated by $R = 1$, while it recovers the exact true tuples for over 98% and 96% of questions in Movie and Restaurant workloads, respectively. In contrast, RAG-EXACT achieves significantly lower eR, recovering the true provenance in only 32% to 68% of cases across the two workloads. Increasing the size of the returned provenance to 10% (RAG-10%) improves recovery but introduces many false positives (over 50 times larger than the ground-truth provenance), making it less interpretable. For example, in Restaurant, RAG-10% returns 106 tuples, while the true provenance contains only 1 tuple, undermining its usefulness. LLM-based baselines recover the exact ground-truth provenance in 50–70% of cases across different LLMs and workloads, and fail to capture the true in over 20% of complex workloads. Finally, the provenance inferred by BLIP can always reproduce the answer, as indicated by $A = 1$, while other approaches have no guarantee of producing verifiable provenance.

Experiment 6: Top-k Provenance. We examine the performance of `top-k-provenance` with k ranging from 1 to 5 in Figure 11. The algorithm stops until *up to* k minimal provenances are found. As k increases, the cost ratio (and similarly, latency) shows a modest increase, rising from approximately 1.3 to 1.8 across all workloads. This is because during the tree-based search in `top-k-provenance`, intermediate results computed for earlier provenances can often be reused when identifying subsequent ones. Consequently, later provenances frequently begin from smaller initial text sequences, leading to lower computation cost and latency. For example, the second provenance starts from node 9's subtree in Figure 7 rather than the full text, reducing cost. Morever, the average size of the returned minimal provenances remains stable, which is consistent with our findings in Table 6.

5.3 User Study

Our experimental evaluation thus far has focused on the accuracy and recoverability of provenance, as opposed to its usefulness. We conducted a user study to assess whether humans find the provenance returned by BLIP to be useful when validating answers from LLMs. We selected four documents from varied domains: *police_reports*, *legal_docs*, *scientific papers*, and *finance_docs*, drawn from our legal partners [2] and open-source benchmarks [4, 20]. We selected three questions per document, testing three scenarios: 1) provenance generated by BLIP; 2) RAG-generated provenance, where up to 5% (or 500 tokens, whichever is smaller) for ease of human interpretation; and 3) no provenance. Our goal was to compare the effectiveness of BLIP's provenance with that returned by RAG. The last scenario was targeted at understanding human review effort to determine correctness when no provenance is provided. We had eight participants in our user study, who considered each scenario for each document, amounting to 12 scenarios in total. We randomized the order of settings across participants.

Metrics. Participants rated the *quality* of a returned provenance from 1 (low)–5 (high), assessing its usefulness in supporting the answer returned by the LLM; we let H_q be the average quality across participants for each dataset. We also asked whether the provenance was both necessary and sufficient (i.e., containing no extra information and missing no critical information); let H_e denote the percentage of such cases, averaged across participants per dataset.

Table 12 reports the average H_q and H_e over eight participants. Overall, participants found the provenance inferred by BLIP to be useful for judging answer correctness, as indicated by the overall H_q of 4.75 (out of 5) across all documents. One participant (P3) commented that "*this seems to contain the right amount of information, including proper context to figure out the answer.*" Dissatisfaction arose mainly from semantic ambiguity between LLM and human interpretations. For example, the provenance "*Use of force details. Involved officer: reporting officer – John Smith*[4]" left it unclear whether John actually used force, suggesting more context could help.

RAG-based provenance was deemed to be less effective, with an overall H_q of 3.1. One participant (P2) complained: "*This looks more like a search result than provenance, and provenance should explain why this answer.*" Participants also found BLIP's provenance to contain the right amount of information 82% of the time, 61% higher than that for RAG-based provenance. When BLIP returned extra information, it was typically unnecessary given users' domain knowledge. For example, users inferred that "*we employed 29,239 people...*" referred to Amazon without the earlier "Amazon" mention BLIP includes—context needed by LLMs but not humans.

In the no-provenance setting, or when the provenance was insufficient (e.g., for RAG), participants often reverted to keyword search. As P7 noted: "*Evidence containing keywords that match the question and answer feels more trustworthy. But I'm not sure if I selected the right keywords due to false positives.*" This keyword search requires multiple passes, typically around 2 distinct keyword search queries, skimming over 18 matches on average to locate relevant content. This inefficiency worsens when keywords are frequently present in the document or when documents are large.

We found two areas of improvement. First, large provenance, even if minimal, was hard to interpret; highlighting keywords

[4]Name anonymized for privacy.

matching the question or answer could help. "*Highlighting sentences within a long provenance and indicating their correspondence to each part of the answer helps a lot*", as noted by P4. Second, providing mechanisms to "tighten" or "expand" the provenance (e.g., dropping the least relevant portions, or including the most relevant excluded portions), depending on the degree of familiarity with the domain, may also be helpful.

6 RELATED WORK

Our work is related to provenance in relational databases, scientific workflows, and LLM inference.

Database Provenance. We compared against database provenance in Section 2.5. Here, we provide a more general background. Database provenance [12, 17, 23, 37] focuses on tracking data lineage to ensure reproducibility. Given a SQL query and inputs, provenance investigates: (1) the input tuples that produce the output (why-provenance) [10, 11, 19], (2) the specific locations (e.g., table, row, or column) in the input from which elements in the output originate (where-provenance) [9, 11], and (3) the transformations applied to the input to derive the output (how-provenance) [25–27, 31]. Our setting poses additional challenges: namely the ambiguity of queries and data, the black-box nature of LLMs, and dealing with LLM costs in inferring provenance.

Provenance in Data Science Pipelines. Provenance has been widely studied in scientific workflow systems, which use a dataflow model. VisTrails [54] enhances provenance query efficiency by enabling users to reuse common sub-workflows. Subzero [62] leverages data locality, where multiple operators in a workflow often share similar input cells. DfAnalyzer [56] tracks provenance at runtime without waiting for workflow completion.Adriane et al. [16] focuses on machine learning workflows, computing provenance to debug model performance. BLIP instead focuses on computing verifiable provenance within a single LLM invocation, and extending BLIP to workflows would be interesting future work.

Provenance in LLM Inference. Overall, LLM provenance has only been studied in limited settings. Prior work, including LLM Attributor [39], ContextCite [18], and Hithesh et al [52] all identify top-k relevant tokens as provenance, using embedding similarities, next-token probabilities, or relevance scores predicted by pretrained models. However, such heuristic methods lack verifiability. Notably, these approaches can be integrated into BLIP as candidate ranking models in our pruning phase. Note that LLM Attributor [39] requires access to LLM training data, which is often unavailable.

7 CONCLUSION

Ensuring trust in LLM-powered data processing tasks is critical for maintaining usability. We presented BLIP, a framework for inferring verifiable minimal provenance—subsets of the input text that reproduce an equivalent answer to the one obtained from the full input. BLIP introduces a set of strategies, each guaranteed to return a minimal verifiable provenance, including an adaptive strategy that combines the strengths of multiple strategies, to infer provenance more efficiently. In scenarios where a single provenance is insufficient to establish confidence in the answer's correctness, BLIP offers a strategy to identify k distinct minimal provenances, enabling users to gain stronger trust. Extensive experiments across five datasets demonstrate that BLIP is guaranteed to return a provenance that can reproduce the answer at a low cost, achieving over 30% higher accuracy than the best-performing baseline.

ACKNOWLEDGMENTS

We acknowledge support from grants DGE-2243822, IIS-2129008, IIS-1940759, and IIS-1940757 awarded by the National Science Foundation, funds from the State of California, funds from the Alfred P. Sloan Foundation, as well as EPIC lab sponsors: Adobe, Google, G-Research, Jane Street, Microsoft, PromptQL, Sigma Computing, Snowflake, and Bridgewater. Compute credits were provided by Azure, Modal, NSF (via NAIRR), and OpenAI.

REFERENCES

[1] 2025. *https://huggingface.co/blog/mlabonne/decoding-strategies.*
[2] 2025. *https://journalism.berkeley.edu/berkeley-and-stanford-police-database/.*
[3] 2025. *https://www.taivo.ai/__are-llms-deterministic/?utm_source=chatgpt.com.*
[4] 2026. *https://www.sec.gov/data-research/sec-markets-data/financial-statement-data-sets.*
[5] Muhammad Adnan, Akhil Arunkumar, Gaurav Jain, Prashant J Nair, Ilya Soloveychik, and Purushotham Kamath. 2024. Keyformer: Kv cache reduction through key tokens selection for efficient generative inference. *Proceedings of Machine Learning and Systems* 6 (2024), 114–127.
[6] Arun Agarwal and Renee Huang. 2025. Introducing Cortex AISQL: Reimagining SQL into AI Query Language for Multimodal Data. https://www.snowflake.com/en/blog/ai-sql-query-language/. Accessed: 2025-06-22.
[7] Rahul K Arora, Jason Wei, Rebecca Soskin Hicks, Preston Bowman, Joaquin Quiñonero-Candela, Foivos Tsimpourlas, Michael Sharman, Meghan Shah, Andrea Vallone, Alex Beutel, et al. 2025. Healthbench: Evaluating large language models towards improved human health. *arXiv preprint arXiv:2505.08775* (2025).
[8] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, et al. 2023. Longbench: A bilingual, multitask benchmark for long context understanding. *arXiv preprint arXiv:2308.14508* (2023).
[9] Deepavali Bhagwat, Laura Chiticariu, Wang-Chiew Tan, and Gaurav Vijayvargiya. 2005. An annotation management system for relational databases. *The VLDB Journal* 14 (2005), 373–396.
[10] Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. 2002. On propagation of deletions and annotations through views. In *Proceedings of the twenty-first ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*. 150–158.
[11] Peter Buneman, Sanjeev Khanna, and Tan Wang-Chiew. 2001. Why and where: A characterization of data provenance. In *Database Theory—ICDT 2001: 8th International Conference London, UK, January 4–6, 2001 Proceedings 8*. Springer, 316–330.
[12] Peter Buneman and Wang-Chiew Tan. 2007. Provenance in databases. In *Proceedings of the 2007 ACM SIGMOD international conference on Management of data*. 1171–1173.
[13] Christopher Buss, Jasmin Mosavi, Mikhail Tokarev, Arash Termehchy, David Maier, and Stefan Lee. 2023. Generating Data Augmentation Queries Using Large Language Models.. In *VLDB Workshops*.
[14] Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Yucheng Li, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Junjie Hu, et al. 2024. Pyramidkv: Dynamic kv cache compression based on pyramidal information funneling. *arXiv preprint arXiv:2406.02069* (2024).
[15] Laura Caspari, Kanishka Ghosh Dastidar, Saber Zerhoudi, Jelena Mitrovic, and Michael Granitzer. 2024. Beyond benchmarks: Evaluating embedding model similarity for retrieval augmented generation systems. *arXiv preprint arXiv:2407.08275* (2024).
[16] Adriane Chapman, Paolo Missier, Giulia Simonelli, and Riccardo Torlone. 2020. Capturing and querying fine-grained provenance of preprocessing pipelines in data science. *Proceedings of the VLDB Endowment* 14, 4 (2020), 507–520.
[17] James Cheney, Laura Chiticariu, Wang-Chiew Tan, et al. 2009. Provenance in databases: Why, how, and where. *Foundations and Trends® in Databases* 1, 4 (2009), 379–474.
[18] Benjamin Cohen-Wang, Harshay Shah, Kristian Georgiev, and Aleksander Madry. 2024. Contextcite: Attributing model generation to context. *Advances in Neural Information Processing Systems* 37 (2024), 95764–95807.
[19] Yingwei Cui, Jennifer Widom, and Janet L Wiener. 2000. Tracing the lineage of view data in a warehousing environment. *ACM Transactions on Database Systems (TODS)* 25, 2 (2000), 179–227.
[20] Pradeep Dasigi, Kyle Lo, Iz Beltagy, Arman Cohan, Noah A Smith, and Matt Gardner. 2021. A dataset of information-seeking questions and answers anchored in research papers. *arXiv preprint arXiv:2105.03011* (2021).

[21] Till Döhmen. 2024. Introducing the prompt() Function: Use the Power of LLMs with SQL! https://motherduck.com/blog/sql-llm-prompt-function-gpt-models/. Accessed: 2025-06-22.

[22] Tianyu Gao, Howard Yen, Jiatong Yu, and Danqi Chen. 2023. Enabling large language models to generate text with citations. *arXiv preprint arXiv:2305.14627* (2023).

[23] Boris Glavic et al. 2021. Data provenance. *Foundations and Trends® in Databases* 9, 3-4 (2021), 209–441.

[24] Google Cloud. 2025. Perform intelligent SQL queries using AlloyDB AI query engine. https://cloud.google.com/alloydb/docs/ai/evaluate-semantic-queries-ai-operators. Accessed: 2025-06-22; Last updated: 2025-06-11.

[25] Todd J Green, Grigoris Karvounarakis, Zachary G Ives, and Val Tannen. 2007. Update exchange with mappings and provenance. VLDB.

[26] Todd J Green, Grigoris Karvounarakis, and Val Tannen. 2007. Provenance semirings. In *Proceedings of the twenty-sixth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*. 31–40.

[27] Todd J Green, Grigoris Karvounarakis, Nicholas E Taylor, Olivier Biton, Zachary G Ives, and Val Tannen. 2007. ORCHESTRA: Facilitating collaborative data sharing. In *Proceedings of the 2007 ACM SIGMOD international conference on Management of data*. 1131–1133.

[28] Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, et al. 2024. A survey on llm-as-a-judge. *arXiv preprint arXiv:2411.15594* (2024).

[29] Neel Guha, Julian Nyarko, Daniel Ho, Christopher Ré, Adam Chilton, Alex Chohlas-Wood, Austin Peters, Brandon Waldon, Daniel Rockmore, Diego Zambrano, et al. 2023. Legalbench: A collaboratively built benchmark for measuring legal reasoning in large language models. *Advances in neural information processing systems* 36 (2023), 44123–44279.

[30] Dan Hendrycks, Collin Burns, Anya Chen, and Spencer Ball. 2021. Cuad: An expert-annotated nlp dataset for legal contract review. *arXiv preprint arXiv:2103.06268* (2021).

[31] Daniel Hernández, Luis Galárraga, and Katja Hose. 2021. Computing how-provenance for SPARQL queries via query rewriting. *Proceedings of the VLDB Endowment* 14, 13 (2021), 3389–3401.

[32] Zezhou Huang, Jia Guo, and Eugene Wu. 2024. Transform Table to Database Using Large Language Models. *Proceedings of the VLDB Endowment. ISSN* 2150 (2024), 8097.

[33] Pranab Islam, Anand Kannappan, Douwe Kiela, Rebecca Qian, Nino Scherrer, and Bertie Vidgen. 2023. Financebench: A new benchmark for financial question answering. *arXiv preprint arXiv:2311.11944* (2023).

[34] Nengzheng Jin, Joanna Siebert, Dongfang Li, and Qingcai Chen. 2022. A survey on table question answering: recent advances. In *China Conference on Knowledge Graph and Semantic Computing*. Springer, 174–186.

[35] Qiao Jin, Bhuwan Dhingra, Zhengping Liu, William Cohen, and Xinghua Lu. 2019. PubMedQA: A Dataset for Biomedical Research Question Answering. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. 2567–2577.

[36] Saehan Jo and Immanuel Trummer. 2024. Thalamusdb: Approximate query processing on multi-modal data. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–26.

[37] Grigoris Karvounarakis, Zachary G Ives, and Val Tannen. 2010. Querying data provenance. In *Proceedings of the 2010 ACM SIGMOD International Conference on Management of data*. 951–962.

[38] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. 2019. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics* 7 (2019), 453–466.

[39] Seongmin Lee, Zijie J Wang, Aishwarya Chakravarthy, Alec Helbling, ShengYun Peng, Mansi Phute, Duen Horng Polo Chau, and Minsuk Kahng. 2025. Llm attributor: Interactive visual attribution for llm generation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 29655–29657.

[40] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.

[41] Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, Benjamin Newman, Binhang Yuan, Bobby Yan, Ce Zhang, Christian Alexander Cosgrove, Christopher D Manning, Christopher Re, Diana Acosta-Navas, Drew Arad Hudson, Eric Zelikman, Esin Durmus, Faisal Ladhak, Frieda Rong, Hongyu Ren, Huaxiu Yao, Jue WANG, Keshav Santhanam, Laurel Orr, Lucia Zheng, Mert Yuksekgonul, Mirac Suzgun, Nathan Kim, Neel Guha, Niladri S. Chatterji, Omar Khattab, Peter Henderson, Qian Huang, Ryan Andrew Chi, Sang Michael Xie, Shibani Santurkar, Surya Ganguli, Tatsunori Hashimoto, Thomas Icard, Tianyi Zhang, Vishrav Chaudhary, William Wang, Xuechen Li, Yifan Mai, Yuhui Zhang, and Yuta Koreeda. 2023. Holistic Evaluation of Language Models. *Transactions on Machine Learning Research* (2023). https://openreview.net/forum?id=iO4LZibEqW Upto date leaderboard at https://crfm.stanford.edu/helm/lite/latest/.

[42] Yiming Lin, Madelon Hulsebos, Ruiying Ma, Shreya Shankar, Sepanta Zeigham, Aditya G Parameswaran, and Eugene Wu. 2024. Towards accurate and efficient document analytics with large language models. *arXiv preprint arXiv:2405.04674* (2024).

[43] Akide Liu, Jing Liu, Zizheng Pan, Yefei He, Reza Haffari, and Bohan Zhuang. 2024. Minicache: Kv cache compression in depth dimension for large language models. *Advances in Neural Information Processing Systems* 37 (2024), 139997–140031.

[44] Chunwei Liu, Matthew Russo, Michael Cafarella, Lei Cao, Peter Baille Chen, Zui Chen, Michael Franklin, Tim Kraska, Samuel Madden, and Gerardo Vitagliano. 2024. A declarative system for optimizing ai workloads. *arXiv preprint arXiv:2405.14696* (2024).

[45] Fenglin Liu, Zheng Li, Hongjian Zhou, Qingyu Yin, Jingfeng Yang, Xianfeng Tang, Chen Luo, Ming Zeng, Haoming Jiang, Yifan Gao, et al. 2024. Large language models in the clinic: a comprehensive benchmark. *arXiv preprint arXiv:2405.00716* (2024).

[46] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics* 12 (2024), 157–173.

[47] Zan Ahmad Naeem, Mohammad Shahmeer Ahmad, Mohamed Eltabakh, Mourad Ouzzani, and Nan Tang. 2024. RetClean: Retrieval-Based Data Cleaning Using LLMs and Data Lakes. *Proceedings of the VLDB Endowment* 17, 12 (2024), 4421–4424.

[48] Linyong Nan, Chiachun Hsieh, Ziming Mao, Xi Victoria Lin, Neha Verma, Rui Zhang, Wojciech Kryściński, Hailey Schoelkopf, Riley Kong, Xiangru Tang, et al. 2022. FeTaQA: Free-form table question answering. *Transactions of the Association for Computational Linguistics* 10 (2022), 35–49.

[49] Liana Patel, Siddharth Jha, Carlos Guestrin, and Matei Zaharia. 2024. Lotus: Enabling semantic queries with llms over tables of unstructured and structured data. *arXiv preprint arXiv:2407.11418* (2024).

[50] Liana Patel, Siddharth Jha, Melissa Pan, Harshit Gupta, Parth Asawa, Carlos Guestrin, and Matei Zaharia. 2024. Semantic operators: a declarative model for rich, ai-based data processing. *arXiv preprint arXiv:2407.11418* (2024).

[51] Matthew Russo, Sivaprasad Sudhir, Gerardo Vitagliano, Chunwei Liu, Tim Kraska, Samuel Madden, and Michael Cafarella. 2025. Abacus: A Cost-Based Optimizer for Semantic Operator Systems. *arXiv preprint arXiv:2505.14661* (2025).

[52] Hithesh Sankararaman, Mohammed Nasheed Yasin, Tanner Sorensen, Alessandro Di Bari, and Andreas Stolcke. 2024. Provenance: A Light-weight Fact-checker for Retrieval Augmented LLM Generation Output. *arXiv preprint arXiv:2411.01022* (2024).

[53] Dario Satriani, Enzo Veltri, Donatello Santoro, Sara Rosato, Simone Varriale, and Paolo Papotti. 2025. Logical and physical optimizations for sql query execution over large language models. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–28.

[54] Carlos E Scheidegger, Huy T Vo, David Koop, Juliana Freire, and Claudio T Silva. 2008. Querying and re-using workflows with VsTrails. In *Proceedings of the 2008 ACM SIGMOD international conference on Management of data*. 1251–1254.

[55] Shreya Shankar, Tristan Chambers, Tarak Shah, Aditya G Parameswaran, and Eugene Wu. 2024. DocETL: Agentic Query Rewriting and Evaluation for Complex Document Processing. *arXiv preprint arXiv:2410.12189* (2024).

[56] Vítor Silva, Daniel de Oliveira, Patrick Valduriez, and Marta Mattoso. 2018. DfAnalyzer: runtime dataflow analysis of scientific applications using provenance. *Proceedings of the VLDB Endowment* 11, 12 (2018), 2082–2085.

[57] Zhaoze Sun, Qiyan Deng, Chengliang Chai, Kaisen Jin, Xinyu Guo, Han Han, Ye Yuan, Guoren Wang, and Lei Cao. 2025. Quest: Query optimization in unstructured document analysis. *arXiv preprint arXiv:2507.06515* (2025).

[58] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel Bowman. 2018. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *Proceedings of the 2018 EMNLP workshop BlackboxNLP: Analyzing and interpreting neural networks for NLP*. 353–355.

[59] Jiayi Wang, Yuan Li, Jianming Wu, Shihui Xu, and Guoliang Li. 2025. Unify: A System For Unstructured Data Analytics. *Proceedings of the VLDB Endowment* 18, 12 (2025), 5287–5290.

[60] Patrick Wendell, Eric Peter, Nicolas Pelaez, Jianwei Xie, Vinny Vijeyakumaar, Linhong Liu, and Shitao Li. 2023. Introducing AI Functions: Integrating Large Language Models with Databricks SQL. https://www.databricks.com/blog/2023/04/18/introducing-ai-functions-integrating-large-language-models-databricks-sql.html. Accessed: 2025-06-22.

[61] Cornelius Wolff and Madelon Hulsebos. 2025. How well do LLMs reason over tabular data, really? *arXiv preprint arXiv:2505.07453* (2025).

[62] Eugene Wu, Samuel Madden, and Michael Stonebraker. 2013. Subzero: a fine-grained lineage system for scientific databases. In *2013 IEEE 29th International Conference on Data Engineering (ICDE)*. IEEE, 865–876.

[63] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W Cohen, Ruslan Salakhutdinov, and Christopher D Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. *arXiv preprint arXiv:1809.09600* (2018).

[64] Bin Zhang, Yuxiao Ye, Guoqing Du, Xiaoru Hu, Zhishuai Li, Sun Yang, Chi Harold Liu, Rui Zhao, Ziyue Li, and Hangyu Mao. 2024. Benchmarking the text-to-sql

capability of large language models: A comprehensive evaluation. *arXiv preprint arXiv:2403.02951* (2024).

[65] Xinyang Zhao, Xuanhe Zhou, and Guoliang Li. 2024. Chat2data: An interactive data analysis system with rag, vector databases and llms. *Proceedings of the VLDB Endowment* 17, 12 (2024), 4481–4484.

[66] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in Neural Information Processing Systems* 36 (2023), 46595–46623.

[67] Jun-Peng Zhu, Peng Cai, Kai Xu, Li Li, Yishen Sun, Shuai Zhou, Haihuang Su, Liu Tang, and Qi Liu. 2024. Autotqa: Towards autonomous tabular question answering through multi-agent large language models. *Proceedings of the VLDB Endowment* 17, 12 (2024), 3920–3933.

[68] Yuchen Zhuang, Yue Yu, Kuan Wang, Haotian Sun, and Chao Zhang. 2023. Toolqa: A dataset for llm question answering with external tools. *Advances in Neural Information Processing Systems* 36 (2023), 50117–50143.